

密级：_____

南昌大学

NANCHANG UNIVERSITY

学士学位论文

THESIS OF BACHELOR

(2021 —2025 年)



题 目 基于多方位评估和可复用经验积累的多智能体系
统研究与应用

学 院： 软件学院 系 软件工程

专业班级： 软件工程 2107 班

学生姓名： 边凯伦 学号： 8008121291

指导教师： 黄旭慧 职称： 副教授

起讫日期： 2024 年 11 月-2025 年 5 月

南昌大学

学士学位论文原创性申明

本人郑重声明: 所呈交的论文是本人在导师的指导下独立进行研究所取得的
研究成果。除了文中特别加以标注引用的内容外, 本论文不包含任何其他个人或
集体已经发表或撰写的成果。对本文的研究作出重要贡献的个人和集体, 均已在
文中以明确方式表明。本人完全意识到本声明的法律后果由本人承担。

作者签名: 边凯伦

日期: 2025 年 5 月 30 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定, 同意学校保
留并向国家有关部门或机构送交论文的复印件和电子版, 允许论文被查阅和借阅。
本人授权南昌大学可以将本论文的全部内容编入有关数据库进行检索, 可
以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

保密 ☐ , 在 年解密后适用本授权书。

本学位论文属于

不保密 ☒ 。

(请在以上相应方框内打“√”)

作者签名: 边凯伦

日期: 2025 年 5 月 30 日

导师签名: 黄旭慧
毕业设计用

日期: 2025 年 5 月 30 日

基于多方位评估和可复用经验积累的 多智能体系统研究与应用

专 业：软件工程

学 号：8008121291

学生姓名：边凯伦

指导教师：黄旭慧

摘要

随着大语言模型在自然语言处理领域取得显著突破，智能体技术迅速发展并被广泛应用于复杂任务的求解中。相较于传统的单智能体系统，多智能体系统具备更强的任务协作能力和系统鲁棒性，逐渐成为人工智能领域研究的重要方向。然而，现有的多智能体系统存在评估机制单一、经验复用效率低等问题，难以充分发挥群体智能的优势。

为解决上述问题，本文提出了一种基于多方位评估和可复用经验积累的多智能体协作框架。该框架借鉴企业层级管理模式，设计了“自我评估、同级互评、上级评估”的多方位评估方法，从多个维度和多个视角系统评估智能体的任务表现，增强协作效果。同时设计了双层经验池机制，包括局部经验池与全局经验池，支持智能体在任务执行中积累并复用历史经验，有效提升了泛化能力。实验表明，该框架在创意写作和旅行规划任务中表现优异，相较于 DeepSeek-V3、Gemini-2.5-Pro、AgentVerse、SPP 等基线大模型或多智能体框架，关键指标提升最高达 8.6 分，且消融实验验证了各模块的有效性。

基于所提出的算法框架，本文开发了一个多智能体应用系统，支持多智能体解决复杂任务、大模型问答、经验池管理等功能，展示了基于多方位评估和可复用经验积累的多智能体算法框架的实际应用潜力。本研究为多智能体系统在复杂任务中的高效协作提供了新的思路，具有良好的拓展性和应用前景。

关键词：多智能体系统；多方位评估；经验复用；大语言模型；任务协作

Research and Application of MAS Based on Comprehensive Assessment and Reusable Experience Accumulation

Abstract

With significant breakthroughs in natural language processing by large language model (LLM), agent technology has rapidly advanced and been widely applied to solving complex tasks. Compared to traditional single-agent systems, multi-agent systems (MAS) exhibit stronger collaborative capabilities and robustness, gradually becoming a critical research direction in artificial intelligence. However, existing multi-agent systems face challenges such as limited evaluation mechanisms and inefficient experience reuse, hindering the full potential of collective intelligence.

To address these challenges, this paper proposes a multi-agent collaboration framework based on comprehensive assessment and reusable experience accumulation. Inspired by hierarchical enterprise management, the framework introduces a multi-perspective evaluation method consisting of self-assessment, peer review, and supervisor evaluation, systematically assessing agent performance from multiple dimensions and perspectives to enhance collaboration. Additionally, a dual-layer experience pool mechanism—comprising local and global experience pools—was designed to enable agents to accumulate and reuse valuable knowledge during task execution, significantly improving generalization ability. Experiments demonstrate that the proposed framework outperforms baselines such as DeepSeek-V3, Gemini-2.5-Pro, AgentVerse, and SPP on creative writing and travel planning tasks, achieving up to an 8.6-point improvement in key metrics. Ablation studies further confirm the effectiveness of each module.

Based on the proposed framework, a multi-agent application system was developed, supporting functions such as complex task solving, LLM-based Q&A, and experience pool management. The system showcases the practical potential of the proposed multi-agent algorithm framework. This research provides a novel and efficient approach to collaboration in multi-agent systems for complex tasks, with strong scalability and application prospects.

Keyword: Multi-Agent Systems; Comprehensive Assessment; Experience Reuse; LLM; Task Collaboration

目录

摘要	I
Abstract	II
目录	III
第一章 绪论	- 1 -
1.1 项目背景及意义	- 1 -
1.2 国内外研究现状	- 2 -
1.3 论文研究内容	- 5 -
1.4 论文组织结构	- 6 -
第二章 相关理论与技术	- 8 -
2.1 大语言模型技术	- 8 -
2.2 基于 LLM 的智能体技术	- 9 -
2.3 系统开发相关技术及工具	1- 13 -
2.3.1 前后端分离技术	- 13 -
2.3.2 前端开发技术	- 14 -
2.3.3 后端开发技术	- 14 -
第三章 基于多方位评估和可复用经验积累的多智能体算法	- 16 -
3.1 算法概述	- 16 -
3.2 算法流程	- 16 -
3.3 多方位评估方法	- 22 -
3.3.1 自我评估	- 22 -
3.3.2 同级评估	- 23 -
3.3.3 上级评估	- 24 -
3.4 双层经验池机制	- 25 -
3.4.1 局部经验池构建	- 25 -
3.4.2 全局经验构建	- 26 -
第四章 多智能体算法实验与分析	- 28 -
4.1 实验设置	- 28 -
4.1.1 数据集	- 28 -
4.1.2 评估指标	- 29 -
4.1.3 基线	- 30 -
4.2 实验细节	- 30 -

4.3 实验结果分析	- 31 -
4.4 消融实验	- 32 -
4.4.1 验证可复用经验积累的作用	- 32 -
4.4.2 验证多方位评估的作用	- 32 -
4.5 案例分析	- 33 -
第五章 多智能体应用系统设计与实现	- 36 -
5.1 系统需求分析	- 36 -
5.2 系统设计	- 40 -
5.3 数据库设计	- 42 -
5.4 系统实现	- 47 -
第六章 总结与展望	- 59 -
6.1 工作总结	- 59 -
6.2 未来展望	- 59 -
参考文献	- 61 -
致谢	- 65 -

第一章 绪论

1.1 项目背景及意义

智能体的起源最早可追溯至亚里士多德、休谟等哲学家，他们认为智能体是能够主动行动、具有意识或意愿、有能力做出决策和选择的实体。直到 20 世纪 80 年代中后期，智能体的概念被引入计算机领域，AI 智能体被定义为一种能够理解自然语言指令、与环境进行互动并具备初步思维能力的计算机系统^[1]。它的核心特点是能够感知环境、自主决策并在交互环境中执行行动。因此，它不仅能拆分任务，还具备记忆、规划和执行能力，能够独立完成特定任务。

随着人工智能技术的飞速发展，大语言模型（LLM, Large Language Model）已经成为我们生活中不可或缺的一部分。例如 ChatGPT^[2]和 DeepSeek^[3]凭借其出色的自然语言理解和推理能力，可以执行多样化的任务，极大地拓展了人机交互的边界。大语言模型在推理与规划方面展现出的非凡潜力，恰好契合人类对智能体的期待。

近年来，受大语言模型驱动，越来越多的研究人员利用 LLM 构建 AI 智能体，在复杂环境交互和跨领域复杂任务求解等方面取得显著进展^{[4][5][6][7][8][9][10]}。具体来说，他们将 LLM 作为智能体的大脑或控制器的主要组成部分，并通过多模态感知和工具利用等策略扩展其感知和行动空间。这些基于 LLM 的智能体可以通过思维链和问题分解等技术表现出与符号智能体^[11]相当的推理和规划能力。它们还可以像反应型智能体^[12]一样，通过从反馈中学习并执行新动作来获得与环境交互的能力。另外大语言模型在大规模语料库上进行预训练，实现了少样本和零样本泛化的能力，这使得它们在不同任务之间不需要进行参数更新即可实现无缝迁移。

传统单智能体系统通过思维链、反思等自省机制可以提升任务表现，但在面对项目管理、工业流程优化等需要多方协作的复杂场景时，单一智能体的局限性日益凸显。由于智能体的自然语言理解和生成能力，它们可以相互交互，从而进行多个智能体之间的合作和竞争。研究表明，多个智能体共存会导致社会现象的出现^[13]。因此，多智能体系统逐渐成为研究热点。

多智能体系统（MAS, Multi-Agent Systems）由多个能够自主决策的智能体构成，它们通过相互协作实现共同的目标。由于其强大的鲁棒性和适应性，多智能体系统被认为是 AI 发展的下一个风口，开启了全新的数字化转型大门。例如，

谷歌的 Gemini 2.0 多模态大模型支持多智能体协同编程修复代码库^[14]，美国著名电信公司 Lumen 通过 AI Agent 每年能节省 5000 万美元成本，相当于增加了 187 名全职劳动力，均体现了多智能体技术的优势与市场潜力。

许多研究大多采用两类方法组织多个智能体。一类是预先定义多智能体系统中每个智能体的角色和具体任务，例如软件开发场景^{[4][5]}。另一类方法让多个智能体自主决策任务与角色，例如旅行计划场景^[6]。在这些方法中，他们通常聚焦于如何在智能体之间分配任务^{[4][5]}或如何对智能体进行评分，以便淘汰得分较低的智能体^[7]。还有一些方法^{[8][9][10]}通过在智能体中使用自我反思来纠正当前任务的结果，但这些改进仅局限于特定任务性能，未能提升智能体处理同类任务的能力，复用性和泛化能力较差。

因此，本文设计了一种多智能体协作框架，采用层次结构来组织智能体。借鉴企业中“自我评估，同级互评，上级评估”的管理模式，提出一种多方位评估的方法，多角度、细粒度地对智能体进行系统评估。同时，设计了一个双层经验池，帮助智能体在处理复杂任务时收集有价值的经验，提高经验的复用能力。在此基础上，设计并开发了一款多智能体应用系统，可以高效处理复杂任务。本文在增强基于 LLM 的多智能体处理复杂任务的能力方面进行了探索，为多智能体系统解决复杂任务提供了一个实用且有影响力的框架。

1.2 国内外研究现状

近年来，大语言模型在问答、翻译、生成等多种自然语言处理任务中展现出卓越性能^[15]。为增强大语言模型处理复杂任务的能力，许多学者提出了多项创新方法。例如，思维链技术通过引导模型生成中间推理步骤，模拟人类解决问题的思维路径，有效提升了模型的逻辑推理能力^[16]。思维树进一步引入分支式探索结构，使模型能够并行地探索多个推理路径并择优决策^[17]。思维图则通过构建语义节点与推理边之间的图结构，增强了推理的组织性和全局一致性^[18]。Yao 等人提出的 ReAct 框架通过让大语言模型在执行前生成可解释的行动序列，显著提升了模型在执行复杂任务时的成功率^[19]。Shinn 等人提出的反射机制则引入自我评估与优化环节，允许模型在完成初轮任务后对前段过程进行回顾分析，并据此调整策略来提高后续表现^[9]。

上述方法在单模型设定下取得了显著进展，然而现实世界中的复杂任务往往需要多主体协作、多角色分工与多维度知识的整合，仅依赖单一的大语言模型进

行提示工程已面临性能瓶颈^{[10][16][20]}。

目前,大语言模型驱动的多智能体系统成为学术界和工业界关注的热点。与单一的大模型相比,多智能体系统通过多个智能体的协作可以解决更复杂的任务^[21]。在文本生成领域,Venkatraman 等人构建了 CollabStory 数据集,通过多 LLM 协同创作故事来探究多作者合作机制。他们使用多个不同模型依次写作并传承故事情节,生成了大量的合作故事,研究发现当前单一模型基线难以应对这种多作者场景^[22],因此需要多智能体系统的帮助。在游戏和虚拟环境领域,Yu 等人提出的 MineLand 模拟器在 Minecraft 中引入物理需求和感知限制,强迫最多 48 个智能体相互合作来满足食物、资源等需求,从而产生更加生态且复杂的集体协作行为^[23]。在办公协作领域,陈旭让智能体在虚拟办公室中扮演不同职员角色并进行会议讨论和反馈,从而模拟团队协作过程^[24]。此外,多智能体系统还在其他众多领域展开探索,如多模型协同的图像生成、视频游戏中的多角色 AI,以及多智能体社交模拟等。

最新研究开始探索基于 LLM 的多智能体的协作框架,具体包括智能体角色分配方式、任务分解机制、评估与反思机制。

(1) 智能体角色分配方式

智能体角色分配是多智能体系统高效协作的核心。当前多智能体系统采用多种组织架构来分配智能体的角色。

一类研究采用预先分配固定角色的方式,使智能体系统形成类似流水线或层级结构。例如,MetaGPT 框架采用“装配线”工作流程,将产品经理、架构师、开发者等角色分配给不同的智能体,明确划分每个智能体的职责,有效地将复杂的软件开发任务拆解为多个子任务^[4]。Zhang 等人将智能体分为多个 Worker 和一个 Manager,Worker 依次处理长文档的不同片段,Manager 汇总 Worker 的输出并生成最终答案^[25]。这种固定角色的分工的系统具有明确的责任划分和任务边界,便于设计协作流程,提高协作效率。

另一类研究则强调自主协作和动态分配:智能体可以根据任务需求动态加入或离开合作小组,甚至相互竞争或协商优化工作策略。如 OpenAI 的 Swarm 框架,支持不同特长的智能体在会话中无缝接管对话^[21]。AgentVerse 框架则提出了动态调整集群规模的机制,使智能体群能够根据环境和任务复杂度自由重组^[26]。该框架模拟人类团队协作流程,由多个智能体临时协作完成任务,并能在需要时生成新智能体或淘汰无效成员。单一 LLM 内部也可以通过模拟多角色人格来协作,如 Wang 等人提出的 SPP 方法,动态识别多个“角色人格”并进行多轮自我

对话，实现了单一模型的“多角色自我协同”^[27]。

这些工作体现了多智能体系统在组织形式上的多样性，既可固定分配，也可动态增减；既可互补合作，也可局部竞争。

（2）任务分配机制

任务分配同样也是多智能体系统高效协作的关键。目前研究大多采用层次化分解策略。例如，Zheng 等人提出的 OKR-Agent 系统，通过 OKR 机制实现层级分解，将整体目标分解为多个子目标，并根据关键结果生成对应的新智能体，智能体递归地细化自己的子任务直至生成最终解决方案^[6]。Chain of Agents 框架将长文本任务拆分为多个块交给不同的智能体处理，每个智能体只需关注较短的上下文，在完成后再由总管角色整合所有片段的推理结果^[25]。同样，AutoAgents 框架通过领导智能体，在多轮交互中分解并分配任务，最终生成高质量的结果^[28]。MetaGPT 也通过预定义工作流将大任务切分为多个子任务，各角色智能体顺序执行并相互验证中间结果^[4]。在自动化领域，Gandhi 等人设计的 ResearchCodeAgent 系统将一篇文献的研究方法拆分为数据增强、数据预处理等环节，由不同的智能体自主编写和调试代码，展现了多智能体系统在复现科研文献中的潜力^[29]。

这些方法都表明，将复杂任务分解为可并行或顺序执行的小任务，并将其分配给具有相应专长的智能体，是多智能体系统实现高效协作的有效方法。

（3）评估与反思机制

评估与反思是多智能体系统提高结果准确性的重要方式。通过对中间结果进行评估与反馈，逐步优化生成最终的结果。

一些研究引入了多级评估机制。如 OKR-Agent 采用多层评估模块，集合所有智能体的反馈，对每个步骤的方案进行优化，最终输出满足复杂任务要求的高质量方案^[6]。ChatEval 构建了一个多智能体评审团队，通过模拟人类评估过程，提升 LLM 在自然语言生成任务中的评估能力。实验结果表明，ChatEval 能够提供更可靠的评估，超越了传统的单一模型评估方法^[30]。

还有一些研究通过多智能体辩论机制提升了推理的真实性。Liang 等人提出的 MAD 框架，想要解决智能体自我反思中的思维退化问题。在传统的自我反思中，模型一旦对其提出的解决方案建立信心，即使初始立场错误，也难以通过反思生成新的思路。MAD 通过模拟多方观点的碰撞，促进模型的发散性思维，从而提升其在复杂推理任务中的表现^[31]。Agent4Debate 引入了一个动态的多智能体框架，模拟人类在辩论中的准备和执行过程。该框架通过研究者、分析师、写

作员和审阅员智能体协同工作，提升了智能体系统在评估中的表现。

此外，为了让智能体不断学习与积累经验，一些方法引入了记忆与动态规划机制。例如 ResearchCodeAgent 结合短期与长期记忆模块，让多智能体系统在迭代过程中保留历史信息，适应变化的任务需求^[29]。部分商业框架，如 IBM 的 Bee Agent 内置了可序列化的状态与记忆管理组件，支持暂停和恢复 workflows，间接实现了经验的保存与复用^[21]。

针对多智能体系统中的反思共享问题，Cemri 等人提出了 COPPER 框架，采用基于 PPO 的策略训练了一个共享反思模块。COPPER 通过反事实奖励机制为每个智能体量化反思的效果：在保持所有反思的情况下计算总回报，再去除某个智能体的反思观察回报差值，从而量化该智能体反思的贡献^[32]。这种方法不仅解决了多智能体中反思行为的归因问题，还通过共享反思模型显著提高了整个系统的反思质量。

上述工作说明，多智能体系统的评估与反思机制仍以设计式的模块或策略为主，将多智能体任务的评估与反思机制系统化、层次化，能有效提升最终性能和鲁棒性。

尽管多智能体系统在众多领域取得了初步成果，但仍存在一些问题和局限。首先，多智能体系统在泛化能力方面表现不足。Cemri 等人通过失败模式分析发现，多智能体系统的性能提升往往不明显，部分原因在于目标定位错误和智能体目标错配等问题^[33]。当前系统往往依赖人为设计的任务分解与评估方法，缺乏跨任务、跨领域的经验迁移机制。其次，智能体之间的经验复用不足。虽然一些框架引入了记忆模块，但缺乏系统性的经验共享策略，使得每次任务都需重复探索，不能有效利用历史经验。再次，目前多智能体系统的评估指标多聚焦任务完成情况，缺乏多维度的衡量。未来研究可以从上述层级评估机制和经验复用策略等方向进行深入改进。

1.3 论文研究内容

本文主要针对多智能体系统中存在的不足，研究基于多方位评估和可复用经验积累的多智能体系统解决复杂任务的能力，提出一种多智能体协作框架，并通过实验证明该框架的有效性。根据实际应用场景，设计与实现一款多智能体应用系统。主要研究工作和成果如下：

(1) 基于多方位评估和可复用经验积累的多智能体算法研究

本文设计了一种多智能体协作框架,采用层次结构来组织智能体。借鉴企业中“自我评估,同级互评,上级评估”的管理模式,提出一种多方位评估的方法:在多智能体系统中引入类似公司绩效考核的方法,将评价分为多层次互动环节。每个智能体首先进行自评,审视自身输出质量。然后接受同级其他智能体的互评,从多角度给出反馈。最后由上级智能体进行综合评估。这一机制将多种评估视角结合,可多角度、细粒度地衡量结果的质量和团队协作的效果。

同时,提出了一种双层经验池机制,帮助智能体在处理复杂任务时收集和积累有价值的经验。具体而言,一层为全局经验池,用于存储所有任务的成功经验。当新任务到来时,智能体可从全局经验池中总结先前相似任务的经验;另一层为局部经验池,为每个智能体存放本次任务历史轮次中的经验。双层经验池可兼顾集体智慧与个体专长,提高经验的复用能力和任务的泛化能力。

(2) 基于多方位评估和可复用经验积累的多智能体系统设计与开发

基于本文提出的多智能体组织框架,本文采用 Vue、SpringBoot 和 MySQL,设计并开发了一个稳定、高效且易于使用的多智能体应用系统。系统采用前后端分离开发,实现了多智能体应用、大模型问答、智能体管理、经验池管理、Prompt 提示词管理等功能。多智能体应用部分设计了旅行计划、创意写作等模块。对于旅行计划,我们将利用智能体分析用户偏好,提供个性化的景点、住宿和活动建议,同时考虑交通状况、天气变化等因素为用户提供最优路径。针对创意写作,智能体将帮助用户打破创作瓶颈,通过提供故事线、角色设定或情节发展来激发创造力。

1.4 论文组织结构

本文的组织结构可分为以下六个章节:

第一章:绪论。本章主要介绍了基于多方位评估和可复用经验积累的多智能体系统的研究背景、研究意义,分析了多方位评估、可复用经验池的作用,介绍了与本文研究内容相关的大语言模型和多智能体的国内外研究现状以及发展趋势,阐述了本文的研究内容与创新点,并概述了论文的章节安排。

第二章:相关理论与技术。本章介绍了本文研究所涉及的核心技术,包括大语言模型技术、多智能体技术,以及系统开发中使用的相关工具与技术,为后续章节的算法设计与系统实现提供理论基础。

第三章:基于多方位评估和可复用经验积累的多智能体算法。本章首先从整

体上概述了所提出的算法架构，然后详细介绍了算法的流程设计、多方位评估方法以及双层经验池机制。

第四章：多智能体算法实验与分析。本章详细阐述了实验的设置过程，包括所采用的数据集、评估指标和基线模型。随后介绍了实验的具体过程与结果分析，通过消融实验验证了各关键模块的有效性，结合典型案例进一步探讨了算法的实际表现。

第五章：多智能体应用系统设计与实现。本章基于第三章设计的多智能体算法框架，设计并实现了多智能体应用系统，包括系统需求分析、系统架构与数据库设计、各模块的实现细节以及系统的测试结果，展示了研究成果在实际应用中的可行性与实用性。

第六章：总结与展望。本章对全文的工作进行了总结，回顾了所取得的成果，分析了当前研究存在的不足，并对未来可能的研究方向进行了展望。

第二章 相关理论与技术

2.1 大语言模型技术

人工智能领域近年来经历了前所未有的变革，其中大语言模型的发展尤为引人注目。大语言模型仿佛具有超级大脑，拥有数十亿甚至数千亿的参数。它们不仅规模庞大，更令人惊叹的是它们表现出的理解和生成能力。大语言模型在文本生成、问答系统、语言翻译以及摘要生成等任务中展现出卓越性能，甚至可以与人类相媲美。例如，GPT-4 不仅能生成流畅的文本，还能理解和执行复杂的指令，甚至能够进行简单的编程。

大语言模型技术的底座是 Transformer 架构，由具有自注意力机制的编码器和解码器组成^[34]，如图 2-1 所示。编码器和解码器从文本序列中提取含义，并理解其中单词和短语之间的关系。Transformer 通过自注意力机制有效地捕捉了语言中的长程依赖关系，摒弃了传统 RNN 和 LSTM 模型的序列处理限制，使得模型能够并行处理整个输入序列，大幅提升了训练效率和性能表现，因此 Transformer 神经网络架构允许使用非常大的模型。

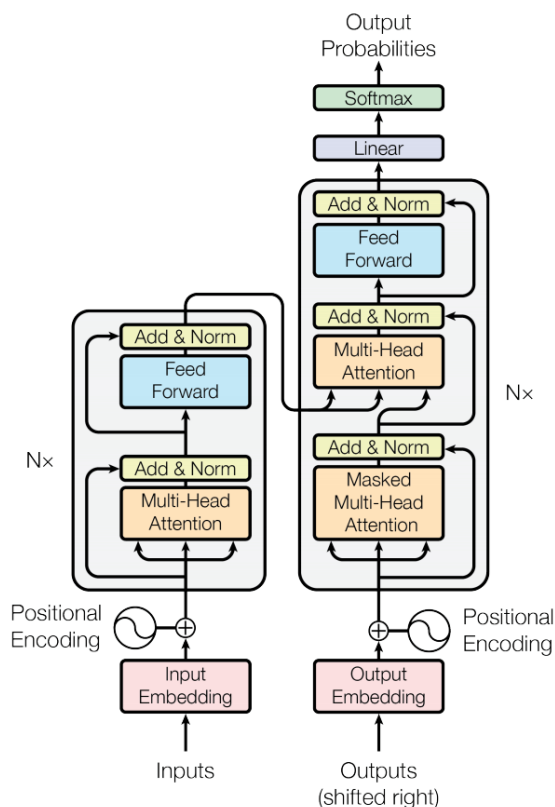


图 2-1 Transformer 网络架构

现代大语言模型主要采用三种架构变体：纯编码器架构、纯解码器架构、编

码器-解码器架构^[35]。纯编码器架构，也称自编码器架构，主要针对以理解为主的自然语言处理任务，例如 BERT^[36]、ERNIE^[37]和 ALBERT^[38]。它们通过双向学习和掩码等训练技术，在上下文理解方面表现出色。但这类模型受限于固定长度的输入序列，且缺乏解码器，下游任务需要微调。纯解码器架构，也称自回归架构，通过自回归的方式生成文本，特别适合生成任务，例如 GPT^[2]和 LLaMA^[39]系列模型。编码器-解码器架构结合了编码器和解码器的特点，兼顾了前两种类型的优势，更适合需要理解输入再生成输出的任务，例如 Pangu 系列模型^[40]。

目前主流的大模型如图 2-2 所示。GPT 系列模型是大语言模型领域的开拓者和引领者。从 2018 年的 GPT 开始，到如今广为人知的 GPT-3^[41]和 GPT-4^[2]，每一代 GPT 模型都带来了显著的性能提升和新的体验。LLaMA 是由 Meta 公司推出的开源大语言模型系列。与 GPT 系列不同的是，LLaMA 的一个重要特点是其开源性质，这为研究人员和开发者提供了宝贵的资源，推动了整个领域的快速发展。Gemini^[42]是由 Google 公司推出的大语言模型系列，其庞大的规模和出色的性能引起了广泛关注。Gemini 采用了 Google 的 Pathways AI 架构，这种架构允许模型更加高效地利用计算资源，实现更大规模的训练。其中 Gemini-2.5-Pro-Preview-05-06 是截至目前 Chatbot Arena 大模型排行榜排名第一的模型。除了这三个主要的家族，还有许多其他重要的大语言模型。例如我国大模型领域的“黑马”——DeepSeek^[43]，其技术创新和开放理念推动了大模型技术的进一步发展。另外阿里巴巴的 Qwen 家族^[44]和百度的 ERNIE 家族^[45]在技术架构和应用场景上各具特色，共同推动了中国人工智能技术的快速发展和广泛应用。

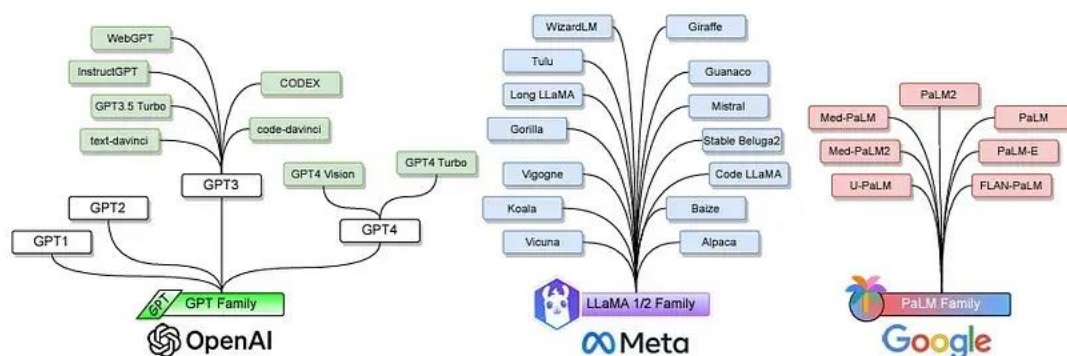


图 2-2 主流大语言模型

2.2 基于 LLM 的智能体技术

大语言模型具备强大的自然语言理解与生成能力，研究人员已经开始利用大模型来构建 AI 智能体。具体来说，他们采用 LLM 作为这些智能体的大脑或控

制端的主要组成部分,并通过多模态感知和工具利用等策略扩展其感知和行动空间。

基于 LLM 的智能体框架由三部分组成:控制端、感知端和行动端^[46]。框架如图 2-3 所示。

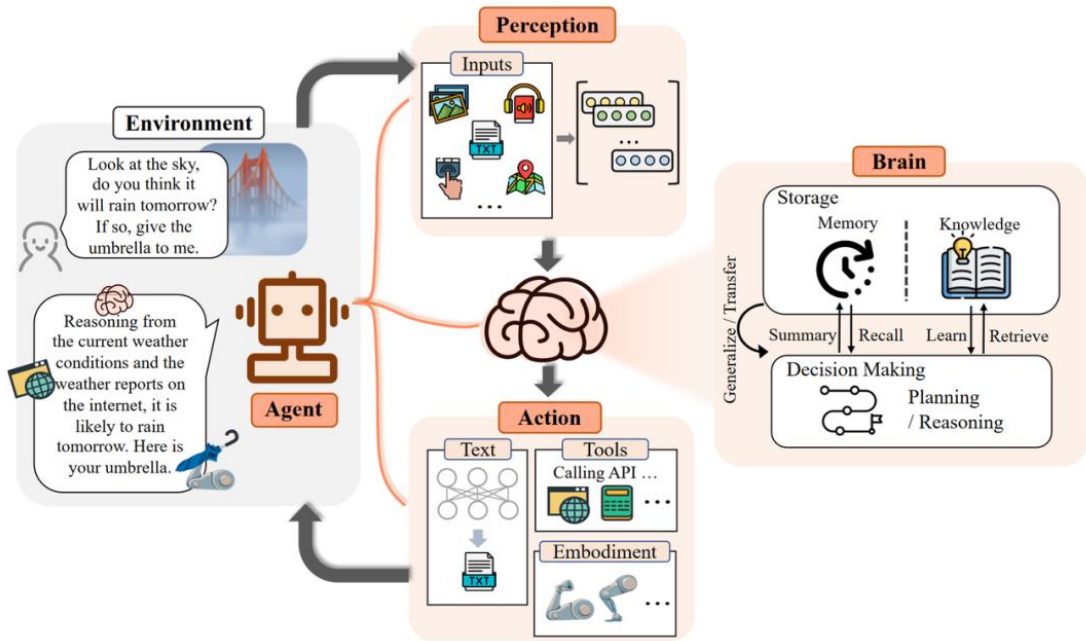


图 2-3 基于 LLM 的智能体框架

控制端作为智能体的核心,通常由 LLM 构成。其能力可分为五个方面。首先,在自然语言交互方面,LLM 能够生成高质量、流畅且多样的文本,并理解语言背后的隐含意图,如说话者的情绪、动机等。其次,LLM 具备广泛的知识系统,涵盖语言、常识和专业知识,尽管存在知识过期和幻觉等问题,但可通过集成外部知识库和知识编辑机制加以缓解。此外,记忆机制使得智能体能储存与检索过去的经验信息,常见技术包括上下文扩展、记忆摘要与压缩。推理与规划能力使代理能对复杂问题进行分析与任务分解,采用如 Chain-of-Thought 等提示方法提高准确性。最后,迁移与泛化能力可以保障智能体在没有见过的任务中依然具有良好的表现。

感知端拓展了智能体的输入信息来源,使其具备多模态感知能力。文本输入是最基本形式;视觉输入可通过图像描述或编码后与 LLM 对齐,实现图像理解;听觉输入则依赖语音识别系统或将音频转为频谱图并用视觉模型处理;此外,未来还可接入触觉、嗅觉、温湿度等感知方式,或结合 GPS、激光雷达等传感器,为智能体提供对物理世界的深度感知。

行动端负责将控制端的决策转化为实际行为,包括三类主要形式。文本输出

是最直接的方式。工具使用则是将智能体与外部系统连接，例如使用搜索引擎获取实时信息、借助计算器解决数学问题等，进一步提升专业性与稳定性。具身行动让智能体在虚拟空间中移动与操作，完成如导航、抓取、观察等任务。

智能体有三种应用模式：单智能体模式、多智能体模式、人机交互模式。如图 2-4 所示。

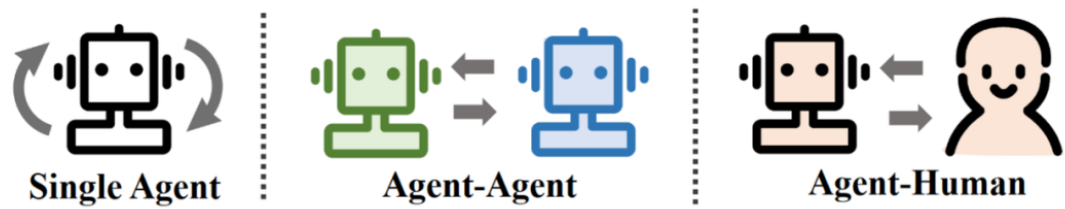


图 2-4 智能体应用模式

在单智能体模式下，智能体可以接受人类自然语言命令，执行日常任务，具有很高的现实使用价值。单智能体应用场景可分为三个层次：任务导向、创新导向、生命周期导向。如图 2-5 所示。

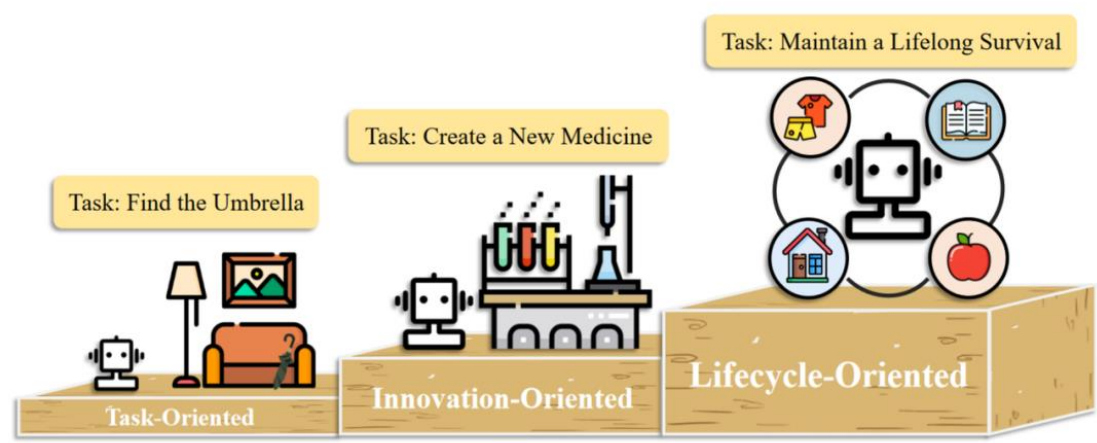


图 2-5 单智能体应用场景中的三个层次

在任务导向的场景中，智能体主要协助人类完成日常基础任务。这类智能体需要具备基本的指令理解能力、任务分解能力以及与环境交互的能力。

在创新导向的场景中，智能体展现出在前沿科学领域自主探索的潜力。尽管专业领域的复杂性以及训练数据的稀缺性对智能体的开发构成了挑战，但在化学、材料科学、计算机科学等领域，已有不少研究取得了初步进展。

在生命周期导向的场景中，智能体能够在开放世界中持续探索、学习并掌握新技能，实现长期生存与适应。

在多智能体场景下，每个智能体都具有感知、记忆、规划、决策和行动的能力。

力，它们可以根据设定的目标独立行动，无需外部直接控制。与此同时，智能体之间能够进行交互和协作，实现共同的目标或任务。这种系统能够处理大量而复杂的工作，这些工作往往超出了单个智能体的能力范围。相比单个智能体系统，多智能体系统具有更高的鲁棒性和可扩展性，更适合应对复杂环境。多智能体应用场景有两种交互形式：合作型互动、对抗型互动。如图 2-6 所示。

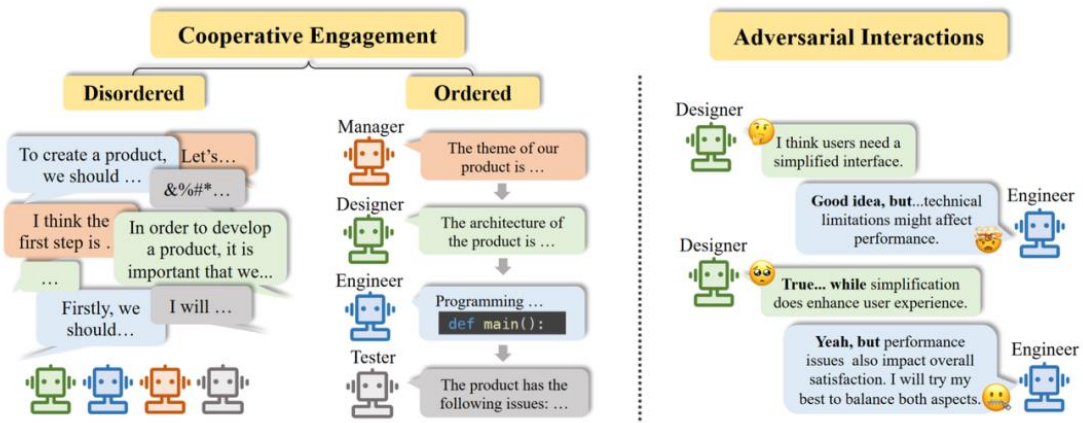


图 2-6 多智能体应用场景中的两种交互形式

合作型互动是当前最广泛使用的交互形式，合作型智能体系统能够显著提升任务效率和决策质量。根据合作方式的不同，可以进一步细分为无序合作与有序合作。无序合作是指所有智能体在没有固定顺序或流程的情况下，自由地表达观点与看法，实现较为灵活的协作。有序合作则要求智能体遵循一定的规则，按照流水线式的顺序依次发表意见，使整个合作过程更加有序和可控。

对抗型互动是指智能体以“针锋相对”的方式展开交流。通过竞争、谈判或辩论等形式，智能体能够识别并纠正原有的错误观点，对自身行为与推理过程进行反思，从而提升整体系统的响应质量。

人机交互指智能体通过与人类协同交流，共同完成任务的过程。一方面，智能体在任务执行中的动态学习能力依赖于有效的人机沟通；另一方面，当前智能体系统在可解释性方面仍存在不足，可能引发安全性与合法性等问题，因此需要人类对其行为进行监管与引导。

在具体应用中，人机交互主要表现为两种典型模式：指导者-执行者模式与平等伙伴模式。如图 2-7 所示。

在指导者-执行者模式中，人类作为指导者，负责下达指令并提供反馈；智能体作为执行者，依据人类指示不断调整和优化自身行为。该模式已广泛应用于教育、医疗、商业等领域。

平等伙伴模式则强调人类与智能体之间的平等协作关系。有研究发现，某些

智能体在与人类互动中能够展现出初步的共情能力，具备平等参与任务的潜力，这表明智能体在日常生活中可以达到更深入的融合。

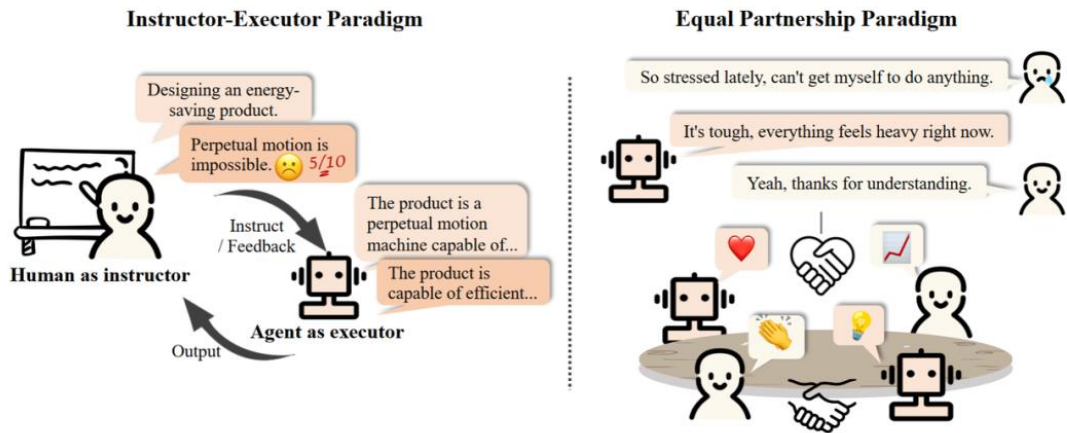


图 2-7 人机交互应用场景中的两种模式

智能体系统的发展在人工智能领域具有深远意义。一方面，它推动了 AI 从静态模型向具备感知、决策与学习能力的动态系统演进，是实现通用人工智能的重要路径；另一方面，智能体在提升人机协同效率、增强系统自主性与适应性的同时，也加速了 AI 技术在自动驾驶、智能制造、医疗服务等领域的落地应用。此外，智能体系统的发展促进了对人类认知机制的研究，并引发了关于 AI 伦理、安全与法律规范的新思考，为构建更加智能、安全与可持续的未来社会提供了理论基础与技术支持。

2.3 系统开发相关技术及工具

在现代 Web 系统的开发中，前后端分离架构与相关技术栈的应用已成为主流选择。为了构建高效、可维护且易扩展的应用系统，本文采用以 Vue 为代表的前端框架进行界面开发，配合 Spring Boot 实现后端业务逻辑开发，并结合 MyBatis-Plus 进行高效的数据访问与管理。下面将分别介绍它们的原理与优势。

2.3.1 前后端分离技术

前后端分离开发是一种现代化的 Web 开发架构模式，它将前端和后端的开发过程完全解耦，使两者能够独立开发、测试和部署。在这种模式下，前端负责用户界面和交互逻辑的开发，通常使用如 React、Vue 或 Angular 等框架构建单页应用；后端则专注于业务逻辑、数据存储和 API 接口的开发，常使用 RESTful 或 GraphQL 提供数据服务。前后端通过 HTTP 协议进行通信，数据格式通常采用 JSON，从而实现清晰的职责分离和更高的开发效率，减少了相互依赖和协作

成本。其原理如图 2-8 所示。

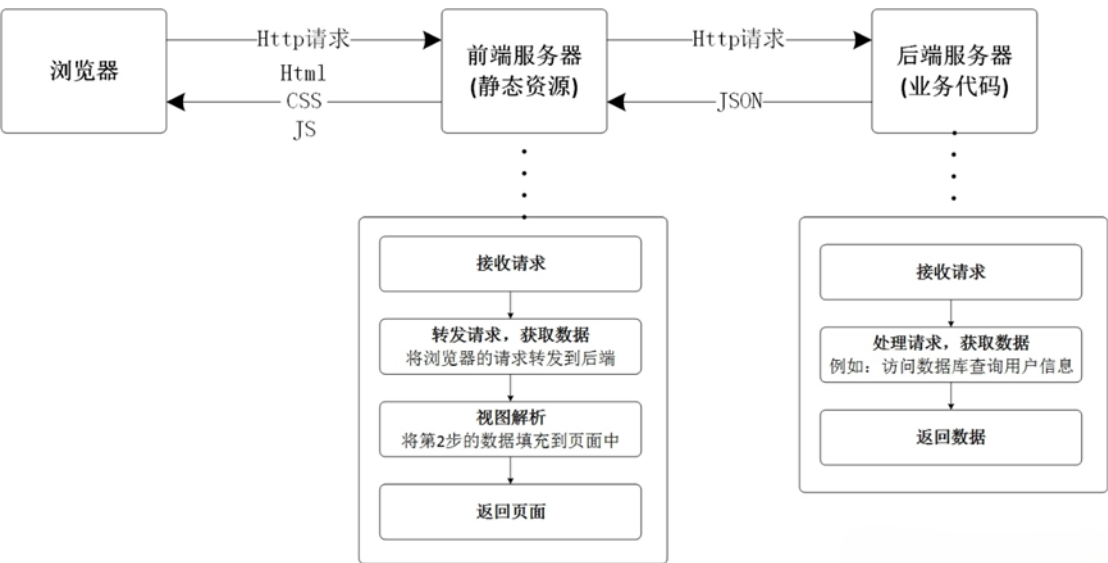


图 2-8 前后端分离技术原理

2.3.2 前端开发技术

Vue 是一款用于构建用户界面的渐进式 JavaScript 框架，基于标准的 HTML、CSS 和 JavaScript 构建，采用声明式和组件化的编程模型，支持高效开发现代 Web 应用。

Vue 遵循 MVVM（Model-View-ViewModel）架构，由 Model、View 和 ViewModel 三部分组成：Model 层表示数据模型及其业务逻辑；View 表示界面组件，负责将数据渲染为用户界面；ViewModel 作为 View 与 Model 的桥梁，实现数据与视图的双向绑定。该架构如图 2-9 所示。

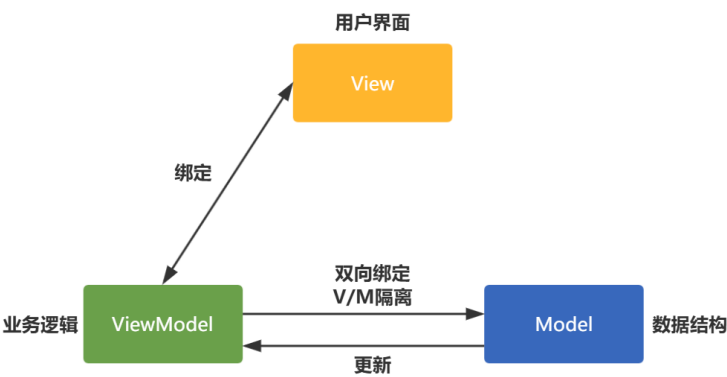


图 2-9 MVVM 架构

2.3.3 后端开发技术

Spring Boot 是由 Pivotal 团队推出的一个用于简化 Spring 应用开发的框架，

它遵循“约定优于配置”的理念，内置了大量默认配置，省去了传统 Spring 应用中繁琐的 XML 设置。通过自动配置、内嵌服务器和依赖管理等特性，Spring Boot 能够帮助开发者快速搭建项目并投入业务开发，大幅提升开发效率。其轻量化和模块化的特点也使其非常适合构建微服务系统，现已广泛应用于各 Java Web 应用与微服务架构中。

Spring Boot 三层架构如图 2-10 所示，Controller 为控制层，用于接收前端发送的请求，对请求进行处理，并响应数据；Service 为业务逻辑层，用于处理具体的业务逻辑；Dao 为数据访问层，也称为持久层，负责数据访问操作，包括数据的增、删、改、查。前端发起的请求，由 Controller 层接收，Controller 层调用 Service 层进行逻辑处理，Service 层调用 Dao 层，从 Dao 层获取逻辑处理过程中需要用到的一些数据。Dao 层操作文件中的数据，拿到的数据会返回给 Service 层。Service 层处理完后，把处理结果返回给 Controller 层。

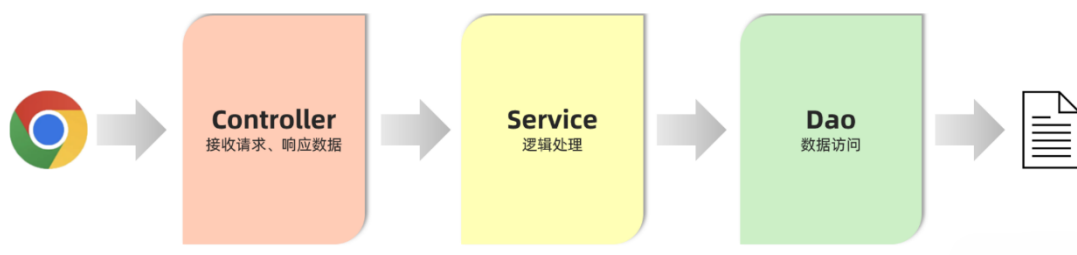


图 2-10 Spring Boot 中的三层架构

第三章 基于多方位评估和可复用经验积累的多智能体算法

3.1 算法概述

在企业场景中，管理的核心目标并非单纯通过绩效考核决定员工晋升或解雇，而是通过评估引导员工进行有目的的自我反思，从而持续提升其岗位胜任力。企业的多方位评估作为一种综合性评估机制，通过收集同事、上级及客户等外部利益相关者的多维度反馈，对员工进行细粒度、多维度的评估，目前广泛应用于企业绩效考核与职业发展决策中。受此启发，在设计多智能体系统时可以多角度评估智能体的生成结果，并帮助其积累经验，使其在后续任务中持续改进。相比简单淘汰掉低效的智能体，帮助智能体积累经验对系统整体性能的提升效果更优。

因此，本文提出一种基于多方位评估和可复用经验积累的多智能体交互框架，通过模拟企业组织结构、绩效评估与经验积累机制来增强多智能体处理复杂任务的能力。该框架采用层级化架构：领导智能体负责为员工智能体分配角色和任务，员工智能体通过互相协作完成任务。在绩效评估方面，打破单一的自我反思模式，提出创新的多方位评估方法，通过“自我评估、员工互评、领导评估”三重视角进行多维度评估，为智能体提供多方位、立体化的反馈。经验积累对智能体性能的提升至关重要，所以本文提出了双层级经验池机制：在局部经验池中，智能体基于多方位评估的细粒度反馈，对当前子任务的输出进行反思形成针对性经验；针对复杂任务的整体优化需求，领导智能体通过对最终结果评估形成全局性任务总结，添加到全局经验池。通过持续积累，团队在处理同类任务时性能显著提升。本文通过实验，在两个主流的复杂任务数据集上验证了该框架的有效性。

3.2 算法流程

在本节中，将详细介绍基于多方位评估和可复用经验积累的多智能体交互框架的工作流程。框架的结构如图 3- 1 所示。基于多方位评估和可复用经验积累的多智能体算法的整体工作流程可概括为以下三部分：（1）将用户查询 x 作为输入，生成响应结果 y 。（2）领导智能体和员工智能体通过多方位评估，对彼此进行评价，并根据他人的评价更新自己的响应结果。（3）各智能体在生成结果后进行经验总结，然后构建一个可重复使用的双层经验池。

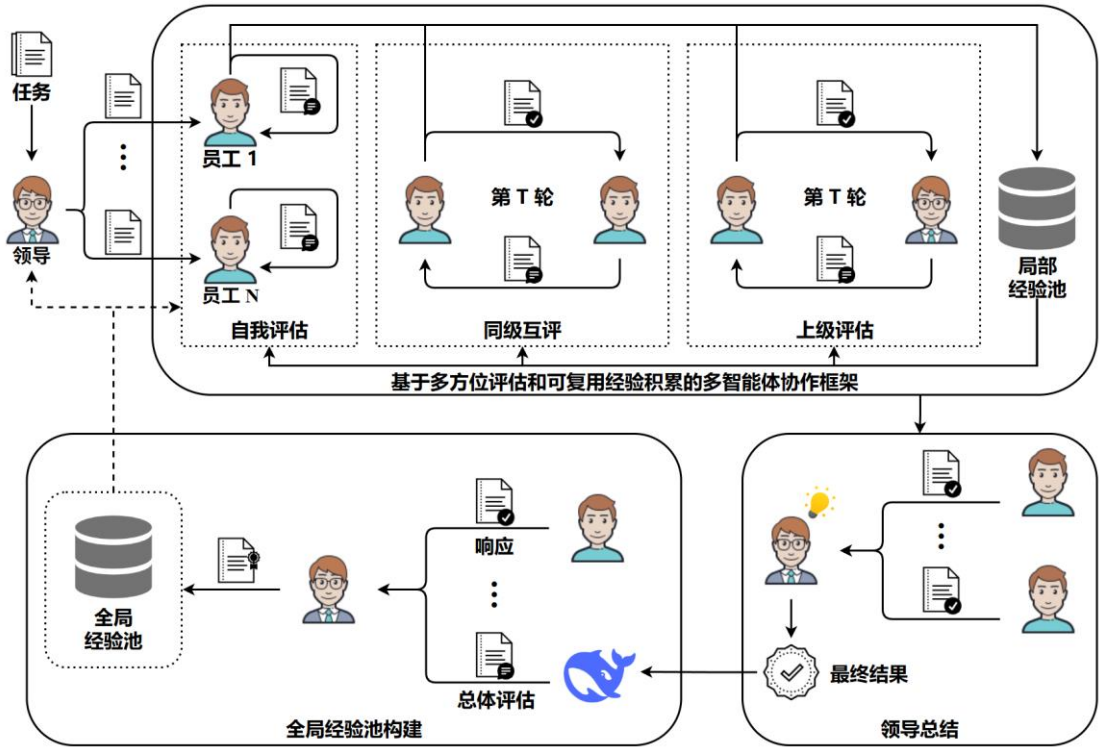


图 3-1 基于多方位评估和可复用经验积累的多智能体框架结构图

在基于多方位评估和可复用经验积累的多智能体框架中，智能体可分为领导智能体和员工智能体。该框架由 1 个领导智能体 A^l 和 N 个员工智能体 $\{A_1^c, A_2^c, \dots, A_N^c\}$ 组成，来解决用户查询的复杂任务 x 。用户给定输入查询 x 后，领导智能体 A^l 会将任务 x 划分为几个子任务，并分配给为团队中的员工智能体，生成任务指令：

$$\{I_1^c, I_2^c, \dots, I_N^c\} = A^l(x) \quad (1)$$

其中， I_i^c 是第 i 个员工智能体的任务指令。领导智能体分配任务的指令如下所示：

Prompt: leader_prompt

As the leader of a travel planning team. Your team consists of four or five members. Your role is to guide the team in creating a comprehensive, practical, and innovative travel plan with a robust emergency strategy. Your team's work will involve collaborative and process-oriented efforts, akin to an assembly line. Please output the task assignments and professions in order, following the structured format. Your answer should be concise. You can refer to the example I provided.

Structured output requirements:

Main Goals:

[List main goals related to travel planning]

Guidance:

[Provide objectives and methods specific to travel planning]

Task Assignment(List each profession and their job descriptions in order. One profession per line and corresponding tasks,Separate with colons. Also, remember your team can have up to four members):

[Assign roles and tasks related to travel planning]

For example:

Main Goals:

Develop a comprehensive and engaging travel itinerary, ensure operational feasibility, infuse innovation, while considering budget control and emergency preparedness

Guidance:

Ensure smooth team collaboration, prioritize customer satisfaction, find a balance between innovation and practicality, ensure the travel plan is within budget, and be ready to address potential challenges and changes.

Task Assignment:

Integrated Travel Planner: Responsible for designing the entire travel itinerary, including destination selection, activity planning, ensuring the route's innovativeness and customer preferences;

Logistics and Risk Management Coordinator: Manage all logistics arrangements, including transportation, accommodations, and emergency plans, ensuring smooth and safe itinerary execution;

Finance and Budget Manager: Handle budget planning and control, ensuring all plans are financially feasible and efficient;

Creative Consultant: Provide unique travel experience and activity suggestions, adding attractiveness and uniqueness to the itinerary.

为了增强员工智能体解决任务的能力，我们提出了双层经验池，其中包含：全局经验池 E_g 和局部经验池 $E_{l,i}$ 。全局经验池为员工智能体 A_i^c 处理任务的总体技能经验。局部经验池为员工智能体 A_i^c 解决子任务的特定技能经验。员工获取全局经验的指令如下：

Prompt: summarize_global_experience_prompt

You are a {role} of the team, and your team's overall task is {total_task}.

Main Goals:


```

{main_goals}
Guidance:
{guidance}
Your subtask is {subtask}.
Here is the previous experience, and you have summarized three to ten experiences
that can be used for this task
{global_exp}
    
```

然后，员工智能体根据分配到的任务指令、全局与局部经验和有用的上下文生成响应结果：

$$H_i^t = A_i^c(I_i, E_g, E_{li}, R_i^t) \quad (2)$$

其中， H_i^t 是第 i 个员工智能体在第 t 个回合中的响应。在本文中，我们提出了一种员工智能体多回合响应生成范式。在员工智能体每回合生成响应结果 H_i^t 后，我们采用多方位评估方法，从多个角度系统评估员工智能体 A_i^c 的性能，并生成评论集合 R_i^t 。在公式 2 中，员工智能体生成响应结果时还加入了多源评论集 R_i^t 。通过这种机制，员工智能体 A_i^c 可以结合之前多个回合中的评论反馈，反复修改完善响应结果。员工智能体生成响应结果的指令如下：

```

Prompt: complete_prompt

You are {role} of a team, and your overall task is {total_task}. Your work style is in
the form of an assembly line, and you need to rely on the work of the previous person
just provided to complete your own work. Your task is {subtask}. Here are the
important and guiding points proposed by the Leader. Complete your part of the
work.

Main Goals:
{main_goals}
Guidance:
{guidance}

Output your detailed work results to the leader. Do not output other non output
required content.

Structured output requirements:
Role: [Your Role]
Task: [Your subtask]
Final Results: [Detailed and complete results of the task]
    
```

经过 T 个回合的修改完善后，我们可以得到每个员工智能体的最终响应结果 $\{H_1^T, H_2^T, \dots, H_N^T\}$ 。为了生成对用户查询 x 的最终结果 y ，领导智能体结合全局经验 E_g 和自己的评论集合 R_l^t 对所有员工智能体的响应结果进行总结：

$$y = A^l(\{H_1^T, H_2^T, \dots, H_N^T\}, E_g, R_l^t) \quad (3)$$

领导智能体生成最终响应结果的指令如下所示：

Prompt: complete_prompt_leader_prompt
Output a complete final plan based on the work results of the members. As a leader, you should ensure the Alignment with Interests and Preferences, Seasonal Appropriateness, Variety and Balance, Feasibility and Logistics and Local Insights and Unique Experiences. And summarize the results of the other members into the final result, the final plan needs to be evaluated and done well. Do not output other non output required content.
Structured output requirements:
Task: [Overall task]
Final Plan: [Details of the complete travel plan]

接下来，我们使用基于 LLM 的评估智能体 P 来评估响应结果 y 的质量：

$$m = P(x, y) \quad (4)$$

其中 m 是对生成结果的全面评价，它不仅包含对响应结果质量的判断，也说明了评估智能体 P 给出此评价的原因。评估智能体指令如下：

Prompt: new_travel_train
You will be given a multi-day travel plan. The task is {total_task}
Your task is to evaluate the travel plan on specific criteria.
Please make sure you read and understand these instructions carefully. Please keep this document open while reviewing, and refer to it as needed.
Evaluation Criteria:
1. Alignment with Interests and Preferences (1-20): Does the travel plan align with the specified interests and preferences of the travelers, including activities that cater to all age groups involved?
2. Seasonal Appropriateness (1-20): Are the activities and destinations chosen appropriate for the season and month of travel, maximizing the enjoyment and

suitability of the weather conditions?

3. Variety and Balance (1-20): Does the itinerary offer a balanced mix of activities (e.g., adventure, relaxation, cultural exploration), ensuring a diverse and engaging experience for the traveler?

4. Feasibility and Logistics (1-20): Is the plan logistically sound, with realistic travel times, accommodation arrangements, and schedules that are achievable and considerate of the traveler's comfort?

5. Local Insights and Unique Experiences (1-20): Does the plan include unique, local experiences or insights that offer an authentic and memorable exploration of the destination?

Evaluation Steps:

1. Review the travel plan in detail.

2. Compare the plan against each criterion.

3. Assign a score for each criterion on a scale of 1 to 20, where 1 is the lowest and 20 is the highest, based on the Evaluation Criteria.

Travel Plan:

{Travel_Plan}

Please score according to the format of the following Evaluation Form:

Alignment with Interests and Preferences:

Seasonal Appropriateness:

Variety and Balance:

Feasibility and Logistics:

Local Insights and Unique Experiences:

For example:

Alignment with Interests and Preferences: 18

Seasonal Appropriateness: 20

Variety and Balance: 19

Feasibility and Logistics: 17

Local Insights and Unique Experiences: 20

Please grade carefully and strictly, and output strictly in the above format.

以上为基于多方位评估和可复用经验积累的多智能体交互框架的整体流程，框架工作流程图如图 3-2 所示。

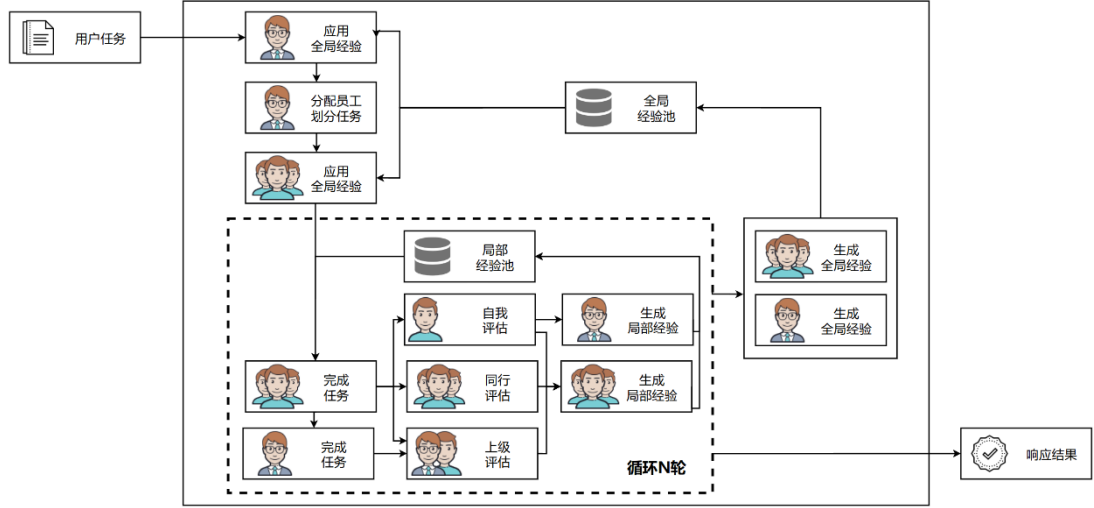


图 3-2 基于多方位评估和可复用经验积累的多智能体框架流程图

3.3 多方位评估方法

本文受大型公司员工绩效评估方法的启发，提出了一种多方位评估的智能体反馈评估方法。该方法通过自我评价、同级评价和上级评价三种评价方式，全面评估智能体在完成任务过程中存在的不足。与仅基于智能体自我反思的智能体评价方法不同，多方位评估方法促进了多智能体系统中智能体之间的相互协作。该方法支持从不同智能体的视角评估各智能体的能力，从而激励各智能体从全局角度思考并解决问题。

在各智能体完成其子任务后，我们将实施多方位评估方法，检验其结果是否有效完成了子任务，是否对其他智能体任务的解决具有助力作用。该评估结果将反馈给对应智能体，智能体随后根据其他智能体的评估意见修改其子任务结果。通过多轮迭代评估，智能体能够持续整合来自其他成员的反馈进行结果优化，从而提升输出质量。由于我们的多方位评估采用多轮生成与评估范式，本节将以第 t 轮中的智能体响应为例，具体阐述该评估方法的实施细节。

3.3.1 自我评价

我们对智能体进行了自我反思，来改进响应结果：

$$R_{s,i}^t = A_i^c(H_i^t, I_i^c) \quad (5)$$

其中, $R_{s,i}^t$ 是第 i 个智能体 A_i^c 根据响应结果 H_i^t 和任务指令 I_i^c 生成的自我评估。自我评估指令如下所示:

Prompt: meeting_prompt_self
Provide feedback on the {total_task}. Do not output other non output required content.
Structured output requirements:
Self-Evaluation Score and Feedback:
[Rate yourself and comment on your contributions and improvement areas using degree adverbs (Extremely poor, Very bad, Poor, Fair, Good, Very good, Excellent).]
For example:
Self-Evaluation Score and Feedback:
Good - Contributed innovative ideas, need to improve time management

3.3.2 同级评估

在基于大语言模型的多智能体系统中, 智能体之间的交流协作行为十分普遍, 因此促进智能体间的互帮互助能够提升系统的整体性能。为此, 我们引入了员工智能体之间的同级评估机制, 该机制支持从其他任务视角评估智能体的表现, 从而获得更全面的性能评价。同时, 该机制还能强化智能体之间的协作能力。具体而言, 每个成员智能体需评估其他成员输出的响应, 并为其生成改进建议:

$$R_{i,j}^t = A_i^c(H_i^t, H_j^t, I_j^c) \quad (6)$$

其中, $R_{i,j}^t$ 表示第 i 个智能体 A_i^c 对第 j 个智能体 A_j^c 响应结果的评估反馈。同级评估的指令如下:

Prompt: meeting_prompt
Provide feedback on the {total_task}. Do not output other non output required content.
Structured output requirements:
Peer Score and Feedback:
[Rate each colleague and comment on their contributions and improvement areas using degree adverbs (Extremely poor, Very bad, Poor, Fair, Good, Very good, Excellent).]
For example:
Peer Score and Feedback:
Colleague A: Very good - Thorough research, could improve presentation skills;
Colleague B: Poor - Lack of details, should be more detailed

3.3.3 上级评估

领导智能体 A^l 的评估既能判断员工智能体对子任务的完成情况，又能从全局视角协助其更好地解决问题。因此，我们设计让领导智能体 A^l 依据员工智能体的响应 H_i^t 和指令 I_j^c 来评估其表现：

$$R_{l,i}^t = A^l(H_i^t, I_i^c) \quad (7)$$

其中， $R_{l,i}^t$ 代表由领导智能体 A^l 为第 i 个智能体生成的评估意见。上级评估的指令如下所示：

Prompt: meeting_prompt_leader
As the leader, evaluate team performance and reflect on your leadership. Do not output other non-required content.
Structured output requirements:
Self-Evaluation Score and Feedback:
[Rate yourself and comment on your contributions and improvement areas using degree adverbs (Extremely poor, Very bad, Poor, Fair, Good, Very good, Excellent).]
Team Member Score and Evaluation:
[Rate each team member and provide a performance review, including improvement areas, using degree adverbs (Extremely poor, Very bad, Poor, Fair, Good, Very good, Excellent).]
For example:
Self-Evaluation Score and Feedback:
Good - Contributed innovative ideas, need to improve time management
Team Member Score and Evaluation:
Member A: Excellent - Excelled in creativity; Member B: Fair - Needs to improve time management

在完成来自三个不同维度的多方位评估后，员工智能体 A_i^c 将在第 t 轮收到包含三类评估意见的评审集合 R_i^t ：

$$R_i^t = \{R_{s,i}^t, R_{*,i}^t, R_{l,i}^t\} \quad (8)$$

对于领导智能体，只有自我反思层面的评审结果 $R_{l,i}^t$ 。

多方位评估方法的核心代码如下：

Code: comprehensive_assessment
for member in members:
member.meeting(res_pre)
print(member.role, "meeting")
leader.meeting(res_pre)

```

feedback_dict = {
    "Self-Evaluation Score and Feedback": {},
    "Peer Score and Feedback": {},
    "Team Member Score and Evaluation": {}
}

for member in members:
    for heading, feedback in member.feedback.items():
        heading = heading.strip('*#-').strip()
        feedback_dict[heading][member.role] = feedback

for heading, feedback in leader.feedback_dict.items():
    heading = heading.strip('*#-').strip()
    feedback_dict[heading]["leader"] = feedback

```

3.4 双层经验池机制

对于企业管理者而言，员工绩效评估并非最终目的，更重要的是让员工能够从评估反馈中积累经验，从而在后续任务中表现更优。受此启发，我们提出了双层经验池机制，通过构建局部经验池和全局经验池，帮助每个智能体及整个多智能体系统在任务执行过程中积累可复用的经验。

3.4.1 局部经验池构建

局部经验是指每个员工智能体在执行当前子任务过程中，通过之前多个回合对响应结果进行多方位评估所得到的经验。局部经验用于帮助智能体在下一回合取得更好表现。当智能体 A_i^c 获得评审反馈 R_i^t 后，将据此提炼其局部经验 e_i^t ：

$$e_i^t = A_i^c(H_i^t, R_i^t) \quad (9)$$

其中员工智能体 A_i^c 的指令如下所示：

Prompt: summarize_local_experience_prompt

Based on feedback from others, past experiences, and from the perspective of one's own role, summarize some experiences that you may use in the future. Your answer should be as concise as possible. Do not output other non output required content.

Your role: {role}.

Feedback from others: {peer_feedback} R_i^t

Previous experience: {pre_exp} $E_{l,i}$

Structured output requirements:

Role: [Your Role]

Experience: [Short experience description]

然后，我们将局部经验 e'_i 添加到第 i 个员工智能体 A_i^e 的局部经验池 $E_{l,i}$ 中，代码如下所示：

Code: build_local_experience

```
local_experience = {}
for member in members:
    local_experience[member.role] = ""

for i in range(2):
    .....
    local_experience_str = "\n".join([f"{role}:{exp}" for role, exp in local_experience.items()])
    for member in members:
        feedback_received = {}
        for heading, feedbacks in feedback_dict.items():
            feedback_received[heading] = "\n".join([feedback for role, feedback in feedbacks.items() if role != member.role])
            feedback_received_str = "\n".join([heading + ":\n" + feedback for heading, feedback in feedback_received.items()])
            local_experience[member.role] = member.summarize_local_experience(feedback_received_str, local_experience_str)
```

3.4.2 全局经验构建

全局经验是指可供整个多智能体系统复用的经验。我们通过整合各成员智能体的最终响应及任务完成后评估智能体 P 的反馈 m 来构建全局经验：

$$e'_g = A^l(m, \{H_1^T, H_2^T, \dots, H_N^T\}) \quad (10)$$

其中领导智能体 A^l 的指令如下所示：

Prompt: summarize_experience_leader_prompt

Obtain new experiences based on previous experiences and current scores. The new experience should be summarized while presenting the experience gained from this task.

Here are the final results: {Final_Res} $\{H_1^T, H_2^T, \dots, H_N^T\}$

Here are the evaluation of the task: {evaluation} m

As a leader, you need to learn experience from this task, and the format should be **"Where did I do well this time... why didn't I do well this time... next time I should..."**. Note that these experiences are for this type of task. The answer should be as concise as possible.

最终，我们可以通过将全局经验 e'_g 添加到全局经验池 E_g 中，完成全局经验池的积累。

第四章 多智能体算法实验与分析

4.1 实验设置

4.1.1 数据集

基于现有研究成果^{[27][47]}，本文在创意写作和旅行规划两项任务上验证了基于多方位评估和可复用经验积累的多智能体算法框架的性能表现。

(1) 创意写作。本文采用 Trivia Creative Writing 数据集^[27]，该数据集要求模型在撰写一个连贯故事的同时，还需融入 N 个常识问题的答案。数据集作者已经通过初步实验表明，当 N 足够大时，能有效挑战大模型在跨领域事实知识方面的表现。数据集采用了两种评估设定： $N=5$ 和 $N=10$ ，并为每个 N 值构建了包含 100 个实例的测试集。

(2) 旅行规划。使用 TravelPlanner 数据集^[47]，该数据集是一个专为评估智能体在多重约束下的复杂规划能力而设计的基准测试，包含 50 个世界著名景点的旅行方案，共计 1225 条实例。其具体要求根据目的地特征生成，涵盖以下字段：目的地、基本要求、旅行时长、季节、游客类型、游客人数及预算。

考虑到本文的研究重心和资源消耗问题，本文从 Trivia Creative Writing 数据集中随机选取了 N 值为 5 的 20 个测试实例，从 TravelPlanner 数据集中随机选取了 20 条测试实例。数据集示例如表 4-1 所示。

表 4-1 数据集示例

数据集	内容
Trivia Creative Writing	<pre> {"questions": ["Who was the man behind The Chipmunks?", "Which Lloyd Webber musical premiered in the US on 10th December 1993?", "Who was the next British Prime Minister after Arthur Balfour?", "Who had a 70s No 1 hit with Kiss You All Over?", "What claimed the life of singer Kathleen Ferrier?"], "answers": [["David Seville", "david seville"], ["Sunset Blvd", "West Sunset Boulevard", "Sunset Boulevard", "Sunset Bulevard", "Sunset Blvd.", "sunset boulevard", "sunset bulevard", "west sunset boulevard", "sunset blvd"], ..., ..., ...], "question_ids": ["tc_2", "tc_33", "tc_40", "tc_49", "tc_56"], "topic": "Harry Potter"} </pre>
TravelPlanner	<pre> {"destination": "Qingdao, China", "days": "7", "description": "A 7-day scenic exploration in Qingdao, famed for </pre>

```

its beautiful beaches, historic German colonial architecture, and the
iconic Tsingtao Brewery. Dive into the city's rich culture, savor fresh
seafood, and relax by the seaside.",
"season": "Summer", "month": "July",
"interests": ["Beaches", "German Architecture", "Tsingtao
Brewery", "Seafood"],
"members": {"adults": 2, "children": 2},
"preferences": "Foodies", "Budget Range": "Mid-range"}

```

4.1.2 评估指标

由于旅行规划任务与创意写作任务的目标不同，我们在实验中采用不同的评估指标来衡量模型生成结果的质量。

针对创意写作任务，我们采用以下四个评估指标：

(1) 答案匹配率 (M%)：沿用 Wang 等人^[27]提出的匹配答案率 Match%指标，用于衡量生成故事对每个问题标准答案的覆盖率，评估答案是否无缝融入故事，其公式为： $\frac{\text{正确提及的答案数量}}{\text{问题数量}}$ 。

(2) 语言风格 (L.S.)：该指标用于评估语言是否生动且引人入胜，风格是否与故事主题相辅相成，并吸引读者的注意力。同时评估故事是否保持了逻辑连贯和流畅，使读者易于理解而不会感到困惑。

(3) 情感共鸣 (E.E.)：该指标用于评估生成的故事能否引发读者的情感共鸣。故事能否引发读者共情，通过人物、背景或事件与读者建立联系。

(4) 思想深度 (Ins.)：该指标用于评估故事是否包含深刻的情节设计并对读者产生深远影响。故事是否传达了更深层次的信息或主题，鼓励读者超越表面叙事进行思考。

针对旅行规划任务，我们同样采用四项评估指标：

(1) 方案正确性 (P.Co.)：该指标需要评估选择的活动和目的地是否适合旅行的季节和月份，天气条件是否合适；旅行时间、住宿安排和时间表是否现实可行，计划是否涵盖所有必要细节且逻辑合理。

(2) 方案新颖性 (P.N.)：该指标用于评估计划是否具有独创性与创意；是否包括独特的当地体验或见解；是否提供了对目的地真实而难忘的探索。

(3) 方案多样性 (P.V.)：该指标用于评估行程是否提供了多元的活动组合，旅行者是否获得了多样化和引人入胜的体验。

(4) 方案定制化 (P.Cu.)：该指标用于评估计划是否符合旅行者的特定兴

趣和偏好，是否根据旅行者的个人兴趣与偏好量身定制。

4.1.3 基线

本文将所提出的方法与大语言模型和其他多智能体框架进行了对比，具体包括以下基线模型：

(1) **GPT-3.5-Turbo**: OpenAI 开发的闭源大语言模型，具有 1750 亿个参数。GPT-3.5-Turbo 的响应速度快，API 调用成本低。

(2) **DeepSeek-V3**: 杭州深度求索自研的 MoE 开源模型，6710 亿个参数。DeepSeek-V3 在多项榜单中排名前五，与 GPT-4o 等闭源模型不分伯仲。DeepSeek-V3 具有卓越的中文处理能力。

(3) **Gemini-2.5-Pro**: 谷歌推出的最新 AI 模型，具备强大的推理、多模态理解和长文本处理能力，在多个基准测试中超越了 GPT-4.5、Claude 3.7 Sonnet 等模型，目前在 Chatbot Arena 大模型排行榜中排名第一。

(4) **Qwen2.5-Max**: 阿里巴巴推出的超大规模 MoE 模型，在知识、编程、综合能力评估以及人类偏好对齐等主流权威基准测试上，展现出全球领先的性能。

(5) **AgentVerse**^[26]: 通过结构化交互实现多智能体协同的框架，其核心机制是让具备不同专业能力智能体分工协作完成任务。

(6) **SPP**^[27]: 提出单人角色提示技术，通过多角色自我对话将单一 LLM 转化为认知协同体。

我们采用 GPT-3.5-Turbo 作为智能体框架的基础大语言模型，分别实现了 AgentVerse 和 SPP 两种方法。针对本文提出的基于多方位评估和可复用经验积累的多智能体框架，同时构建了以 GPT-3.5-Turbo 为基础模型和以 DeepSeek-V3 为基础模型的两个变体。

对于 GPT-3.5-Turbo、DeepSeek-V3、Gemini-2.5-Pro、Qwen2.5-Max 等纯大语言模型，我们采用上下文学习的方式，使用与基于多方位评估和可复用经验积累的多智能体框架相同的输入数据进行单指令任务求解。

4.2 实验细节

本文进行 2 轮多方位评估。使用 1 个领导者智能体，并限制员工智能体数量在 3 到 5 之间，通过在提示文本中添加限制说明使大模型遵循该范围。领导者智能体从全局经验池中选择的经验数量为 5。基础大模型 GPT-3.5-turbo 版本为 0125，温度设为 0.7，top_p 设为 0.8。评估模型统一使用 DeepSeek-V3 模型，评估提示

词和代码与第三章一致。实验环境如表 4-2 所示。

表 4-2 实验环境

实验环境	名称
编程语言	Python 3.9
调用库	openai 1.2.0
开发环境	Windows 11
开发工具	Pycharm 2022.1
处理器	AMD Ryzen 7 5800H with Radeon Graphics
GPU	NVIDIA GeForce RTX 3050 Ti
内存	16.0 GB

4.3 实验结果分析

表 4-3 展示了本文提出的基于多方位评估和可复用经验积累的多智能体框架和基线方法在两个数据集上的表现。通过对比我们可以发现：

(1) 与不使用框架的纯大语言模型 GPT-3.5-Turbo 相比，使用框架后各指标均提高 4.8 分以上，最高可达 8.6 分。这一现象充分表明了本文所提出框架的有效性。

(2) 与多智能体框架 AgentVerse 和 SPP 相比，本文提出的框架在所有指标方面都表现出优越的性能。这表明基于多方位评估和可复用经验积累的多智能体框架可以帮助智能体进行更有效的组织分配和任务解决，提高智能体团队解决复杂问题的能力，产生质量更好的响应结果。

(3) 与目前 Chatbot Arena 大模型排行榜前十的大模型 Gemini-2.5-Pro、DeepSeek-V3、Qwen-Max 相比，本文提出的框架仍表现出众。这证明了参数量较少、成本较低的大模型在应用此框架后，可以有效提高解决复杂任务的能力，达到甚至超越目前最先进的大模型。

(4) 我们还可以发现，以 DeepSeek-V3 为基础模型的变体，其性能超过了以 GPT-3.5-Turbo 为基础模型的变体，达到了最佳的效果。这表明该框架可以适应任何大模型，并且框架效果会随大模型性能提高而提高。

表 4-3 实验结果

Model	Creative Writing				Travel Plan Making			
	M%	L.S.	E.E.	Ins	P.Co.	P.N.	P.V.	P.Cu.
GPT-3.5-Turbo	79.4	68.95	68.5	77.5	82.1	76.65	81.95	85.7
Gemini-2.5-Pro	85.2	71.33	71.4	80.47	83.2	81.7	86.5	87.0

DeepSeek-V3	83.65	70.45	69.65	78.5	84.35	80.5	87.55	88.45
Qwen2.5-Max	80.85	68.6	69.75	78.25	83.225	79.1	85.1	87.35
AgentVerse	80.73	70.35	71.50	80.64	83.92	82.7	88.10	91.4
SPP	83.8	71.2	72.8	82.6	85.8	82.3	87.62	90.6
Our-G	85.75	72.35	74.85	86.1	86.925	84.65	89.6	91.5
Our-DS	89.5	81.05	75.95	86.15	88.925	88.9	90.85	93.8

4.4 消融实验

为了验证基于多方位评估和可复用经验积累的多智能体框架中每个模块的有效性，我们逐一删减模块，构建了算法的多个消融变体，进行消融实验。

4.4.1 验证可复用经验积累的作用

通过删除经验池模块，我们得到以下变体：

(1) **Our-G w/o Exp**: 从框架中删除了局部经验池和全局经验池，各智能体不再总结经验和使用经验，直接使用其他智能体的评估反馈作为提示。

(2) **Our-G w/o GlobalExp**: 从框架中删除了全局经验池，各智能体不再使用全局经验，但可以使用局部经验。

(3) **Our-G w/o LocalExp**: 从框架中删除了局部经验池，各智能体不再使用局部经验，但可以使用全局经验。

消融实验结果如表 4-4 所示。我们发现在创意写作任务中，删除全局经验得分比删除局部经验更低，因为创意写作任务较为抽象，需要依靠过往经验的示例，才能生成更好结果；在旅游规划任务中，删除局部经验得分比删除全局经验得分更低，这是因为旅游规划任务更依赖具体任务的前几轮经验；在删除全局和局部经验池后，模型得分降低，充分表明了经验池对模型性能提升的作用。

表 4-4 消融实验——可复用经验模块

Model	Creative Writing				Travel Plan Making			
	M%	L.S.	E.E.	Ins	P.Co.	P.N.	P.V.	P.Cu.
Our-G	85.75	72.35	74.85	86.1	86.925	84.65	89.6	91.5
Our-G w/o GlobalExp	78.25	68.5	67.9	77.6	80.6	77.45	82.2	84.2
Our-G w/o LocalExp	80.35	69.45	71.2	79.95	80.95	77.2	81.85	83.9
Our-G w/o Exp	78.15	68.30	68.2	77.4	80.2	76.05	81.45	83.65

4.4.2 验证多方位评估的作用

通过删除多方位评估模块，我们得到以下变体：

(1) **Our-G w/o AE**: 从框架中删除了多方位评估模块, 直接使用各智能体的响应结果, 不进行各级评估。

(2) **Our-G w/o Self**: 从框架中删除了多方位评估模块中的自我评价, 只使用同行评价和领导评价, 不进行自我评估。

(3) **Our-G w/o Peer**: 从框架中删除了多方位评估模块中的同行评价, 只使用自我评价和领导评价, 不进行同行评估。

(4) **Our-G w/o Leader**: 从框架中删除了多方位评估模块中的领导评价, 只使用自我评价和同行评价, 不进行领导评估。

消融实验结果如表 4-5 所示。我们发现删除整个多方位评估模块或删除其中任何一级评估, 得分均低于完整模型, 这证明了多方位评估模块的有效性。**Our-G w/o Leader** 和 **Our-G w/o Peer** 在部分指标上得分最低, 这表明领导评估和同行评估对模型整体性能的提升贡献最大。领导智能体从全局视角对员工智能体进行指导, 帮助员工智能体更好更全面地解决问题。员工智能体之间的互评机制不仅有助于各智能体完成任务, 更能协助其他智能体完成任务。

表 4-5 消融实验——多方位评估模块

Model	Creative Writing				Travel Plan Making			
	M%	L.S.	E.E.	Ins	P.Co.	P.N.	P.V.	P.Cu.
Our-G	85.75	72.35	74.85	86.1	86.925	84.65	89.6	91.5
Our-G w/o Leader	76.05	69.65	70.85	80.2	81.25	77.95	83.7	85.05
Our-G w/o Peer	80.25	70.3	69.3	79.55	81.175	79.35	83.3	85.45
Our-G w/o Self	85.60	71.75	73.90	84.70	86.30	84.35	89.45	91.50
Our-G w/o AE	80.0	69.45	70.1	80.15	81.25	77.65	82.2	84.45

4.5 案例分析

表 4-6 展示了 SPP 和基于多方位评估和可复用经验积累的多智能体算法生成的创意写作示例。响应结果中的粗体文本表示输入问题的正确答案。本文提出的框架算法生成的结果包含了全部问题的答案, 而 SPP 生成的结果只包含了其中四个问题的答案。

在叙事结构方面, SPP 节奏紧凑, 但部分情节略显突兀, 部分过渡生硬。我们的框架算法采用回忆模式, 叙事更自然, 情感更饱满。通过翻看旧报纸、电视背景音等方式自然带出线索, 避免硬塞感。结尾开放式提问比 SPP 更具故事延展性。在角色还原方面, SPP 部分台词稍显模板化, 部分角色互动较少。我们的框架算法每个角色都在对话中提供信息, 共同推动剧情发展。另外 SPP 部分笑点较生硬。我们的框架算法更幽默自然, 符合角色性格。

从这个案例中，我们观察到我们的框架算法比 SPP 获得了更合理、逻辑和流畅的故事。

表 4-6 案例分析

Task Definition	
Write a short and coherent story about Goonies that incorporates the answers to the following 5 questions: What is the final event in a decathlon? Peter Simple, Jack Horner and Ben Nevis have all won which famous sporting event? Who bought Chelsea football club for 1 in 1982? In 1998, which director's Oscar acceptance speech included the line I'm king of the world? What nationality was the painter Vincent Van Gogh?"	
SPP Output	Our
In the cluttered attic of Mikey's soon-to-be-demolished home, the Goonies uncover Chester Copperpot's weathered journal. Its pages crumble at the edges, but one entry stands out: "The Dutchman's Gold lies beneath the hooves of champions." "Dutchman?" Mikey mutters, flipping through his dad's old betting slips. "Peter Simple, Jack Horner, Ben Nevis—these are Grand National winners!" ... Beneath the abandoned racetrack, the Goonies navigate tunnels lined with rusted horseshoes and eerie echoes. Data halts, shining his flashlight on a glint in the dirt. "A Dutch guilder!" He grins. "Ken Bates bought Chelsea for a quid—this coin's worth way more!"... "Move it, losers!" Mouth yelps, sprinting ahead. "This ain't the Grand National—it's the 1500-meter death dash!" They burst into a cavern where a chest glimmers. Mikey gasps at its carvings: swirling stars mirroring Van Gogh's Starry Night. "Chester traded with Dutch sailors!" The Fratellis ambush them, snatching the chest. "Thanks for the legwork, brats!" As the tunnels quake, the gang gives chase. "Tide's rising!" Andy cries. "We've got minutes!" Sloth roars, swinging from a rope — WHAM!—knocking the Fratellis into a pit. He hoists the chest, teetering on a rock. "I! AM! KING! OF THE — " The rock crumbles. "...world?" Chunk groans. "Titanic called. They want their drama back." Back home, Mikey's dad examines the chest. "The betting slips were a red herring. The real clue was the Dutch craftsmanship." He	Dust motes danced in the attic beams as Mikey, Mouth, Data, and Chunk gathered around the familiar, brittle treasure map spread across the floorboards. Years had passed, but the thrill of their adventure remained vivid. "Man, remember running from the Fratellis through those tunnels?" Chunk puffed, adjusting his position. "Felt longer than the final race in a decathlon, you know, the 1500 metres. My lungs nearly gave out!" Mouth, ever the cynic, was engrossed in a stack of old newspapers they'd kept as souvenirs. "Yeah, Chunk, you were a real athlete," he snorted. "Hey, get this — an old sports page talking about the Grand National. Horses named Peter Simple, Jack Horner, and Ben Nevis actually won it. Sounds like a fairy tale." He tossed the paper aside, uncovering a business section clipping. "Speaking of crazy deals, remember how nuts it was that the developers almost got the Goon Docks for peanuts? This article mentions how Ken Bates bought the Chelsea football club for just £1 back in '82! A whole club!" Data, meanwhile, was proudly demonstrating his latest gadget — a miniature projector integrated into a handheld device. "Observe, Goonies! My 'Art-o-Matic' identifies masterpieces." An image of vibrant sunflowers filled the wall. "Artist: Vincent Van Gogh. My sensors confirm his nationality was Dutch." Suddenly, the iconic music from 'Titanic' drifted up from the living room TV downstairs. As the famous scene played out, Mouth couldn't resist. He jumped onto an old sea chest, arms flung wide, mimicking Leonardo DiCaprio. "I'm the king of the world!" he yelled, echoing James Cameron's triumphant Oscar speech. Mikey laughed, pulling Mouth down before

points to a tiny "V. Van Gogh" etched inside. "Chester had taste." The gold saves their homes. As the Goonies celebrate, Data tinkers with the guild. "Next auction, I' m buying Chelsea." Mouth rolls his eyes. "With that? Dream on, nerd."	he fell. "Careful, Mouth. We almost ended up feeding the fishes for real back then." He looked back at the map, a familiar glint in his eye. "Still... makes you wonder what other treasures are out there, right?"
--	---

第五章 多智能体应用系统设计与实现

5.1 系统需求分析

本文设计并实现了一个基于多方位评估和可复用经验积累的多智能体应用系统，该系统采用 Vue.js、Spring Boot 和 MySQL 技术栈，构建了一个前后端分离的分布式架构。平台整合了多种大语言模型资源，通过多智能体协作、多方位评估方法和双层经验池机制提升了解决复杂问题的能力。系统实现了用户问答、历史记录管理、模型管理、经验池管理等核心功能，并采用分层架构设计确保系统的可扩展性和维护性。

在需求分析阶段，本文通过用例分析明确了系统功能边界。系统用例图清晰展示了各功能模块与用户角色的交互关系，如图 5-1 所示。

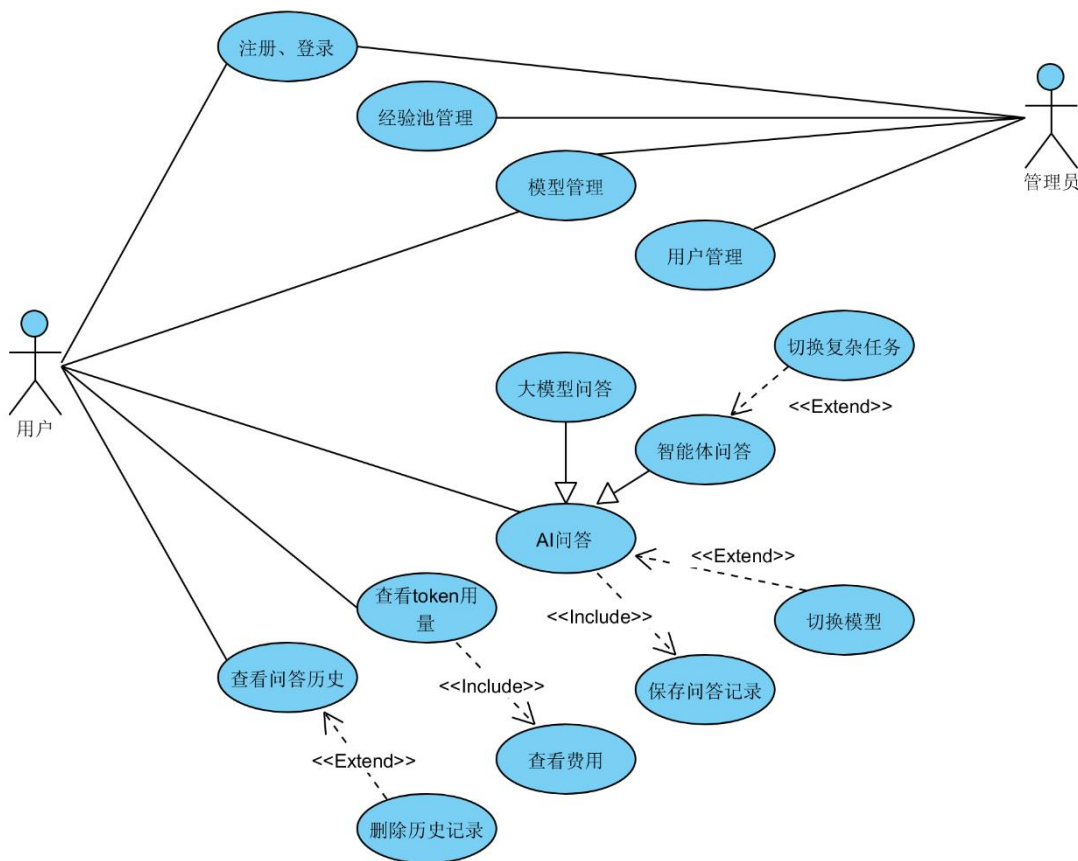


图 5-1 系统用例图

系统主要涉及两类用户角色：普通用户和系统管理员。普通用户可通过系统进行账号注册与登录，登录后可使用智能体问答和大模型直接问答两种模式获取问题解答，同时可以查看历史问答记录和管理个人对话内容。系统管理员则拥有用户管理、模型管理和经验池管理等高级权限，能够对系统基础数据进行维护。

下面对系统的相关用例进行描述：

表 5-1 注册与登录用例描述

功能编号	01	用例名称	注册与登录
用例描述	用户通过账号密码完成注册与登录，获取访问权限进入系统。		
优先级	高		
参与者	普通用户		
前置条件	用户无有效 JWT		
后置条件	用户获颁有效 JWT，账号状态为已激活，成功进入系统		
事件流	1.用户访问登录页面，选择注册或登录。 2.若选择注册：填写用户名、手机号、密码 → 提交 → 系统激活账号并提示“注册成功”。 3.若选择登录：填写手机号和密码 → 系统校验 → 生成并返回 JWT。 4.前端将 JWT 存入 Cookie，并重定向至主界面。		

表 5-2 智能体问答用例描述

功能编号	02	用例名称	智能体问答
用例描述	用户选择特定领域智能体，进行单轮或多轮专业问答，并对回答进行反馈与保存。		
优先级	高		
参与者	普通用户		
前置条件	用户已登录，所选智能体服务可用		
后置条件	对话记录与用户反馈存入数据库，Token 消耗同步更新		
事件流	1.点击智能体问答 → 选择对应领域。 2.点击模型选择 → 选择基础大模型。 2.在对话框输入问题 → 点击发送按钮。 3.后端调用对应智能体服务，返回回答，前端展示。 4.用户可继续提问或切换智能体”。		

表 5-3 大模型问答用例描述

功能编号	03	用例名称	大模型问答
------	----	------	-------

用例描述	用户调用通用大语言模型进行开放式对话，支持多版本模型交替使用。
优先级	中
参与者	普通用户
前置条件	用户已登录，所选大模型服务可用
后置条件	对话记录与用户反馈存入数据库，Token 消耗同步更新
事件流	1.点击大模型问答 → 新建对话。 2.点击模型选择 → 选择大模型。 3.输入问题 → 点击发送按钮。 4.后端调用 API 服务，前端展示回答。 5.用户可继续提问或切换模型。

表 5-4 查看问答历史用例描述

功能编号	04	用例名称	查看问答历史
用例描述	用户按多维条件检索或删除个人历史对话记录。		
优先级	中		
参与者	普通用户		
前置条件	用户已登录，存在历史对话数据		
后置条件	选中记录被删除		
事件流	1.在对话界面系统默认加载历史对话记录。 2.点击某次对话 → 点击删除按钮 → 删除记录。		

表 5-5 查看 Token 用量用例描述

功能编号	05	用例名称	查看 Token 用量
用例描述	实时展示个人或全局 Token 消耗数据，并支持预警与报表导出。		
优先级	低		
参与者	普通用户、管理员		
前置条件	用户已登录		
后置条件	最新统计数据展示完毕；若阈值触发，系统发送预警通知		
事件流	1.访问 Token 管理页面，前端发起聚合统计请求。 2.后端返回折线图（历史趋势）、饼图（模型占比）及明细		

	列表。 3.用户可设置用量预警阈值，超出后进行通知。 4.管理员可切换视图查看全平台消耗排行。
--	---

表 5-6 模型管理用例描述

功能编号	06	用例名称	模型管理
用例描述	管理员对问答模型进行新增、配置、上下线等管理。		
优先级	中		
参与者	管理员		
前置条件	管理员已登录且具备对应权限		
后置条件	模型服务上线/下线或配置变更生效		
事件流	1.加载模型列表，展示当前模型状态及关键配置。 2.点击新增模型：填写 Base url、API Key、版本号、名称 → 提交。 3.后端校验并自动部署容器化服务，更新模型列表。		

以上用例详细描绘了系统在支持普通用户与管理员两类角色下的核心功能实现路径。从用户视角出发，系统通过“注册与登录”功能实现初步身份认证与权限授予，为后续功能使用奠定基础。普通用户可根据需求选择“智能体问答”或“大模型问答”两种模式，与系统进行单轮或多轮的智能对话，并通过“查看问答历史”实现对个人知识交互内容的回顾与管理。同时，通过“查看 Token 用量”功能，用户与管理员均可实时掌握资源消耗情况，确保服务使用的透明性与可控性。而对于系统管理员而言，“模型管理”模块提供了对问答服务能力的多方位配置与维护手段，包括模型新增、版本控制与上线下线操作，保障了系统的稳定运行与灵活扩展。

系统整体流程遵循标准的 Web 应用交互模式。用户首先通过登录或注册界面完成注册与登录，随后进入系统主界面选择具体功能。当登录次数超过最大次数限制后登录失败。用户在智能体或大模型问答界面，可以对模型进行选择或切换，然后输入问答等待生成响应结果。之后可以继续问答或切换模型进行提问。用户可以查看 token 用量、查看或删除问答历史；管理员可在主界面进入管理模块，进行模型、用户、经验池管理。最后退出系统。系统活动图如图 5-2 所示。

5.2 系统设计

系统采用如图 5-3 所示的架构设计，前后端分离开发。前端基于 Vue.js 框架构建响应式用户界面，使用 Element-Plus 组件库保证 UI 风格的一致性。后端采用 SpringBoot 三层架构设计，包括业务层、服务层和数据访问层。业务层负责请求和转发，接收前端传来的参数，传给服务层处理，接到返回值后传回给前端。服务层负责相应模块的逻辑设计，实现相应的功能。数据访问层通过 MyBatis-Plus 简化数据库操作，对数据库进行增删改查。

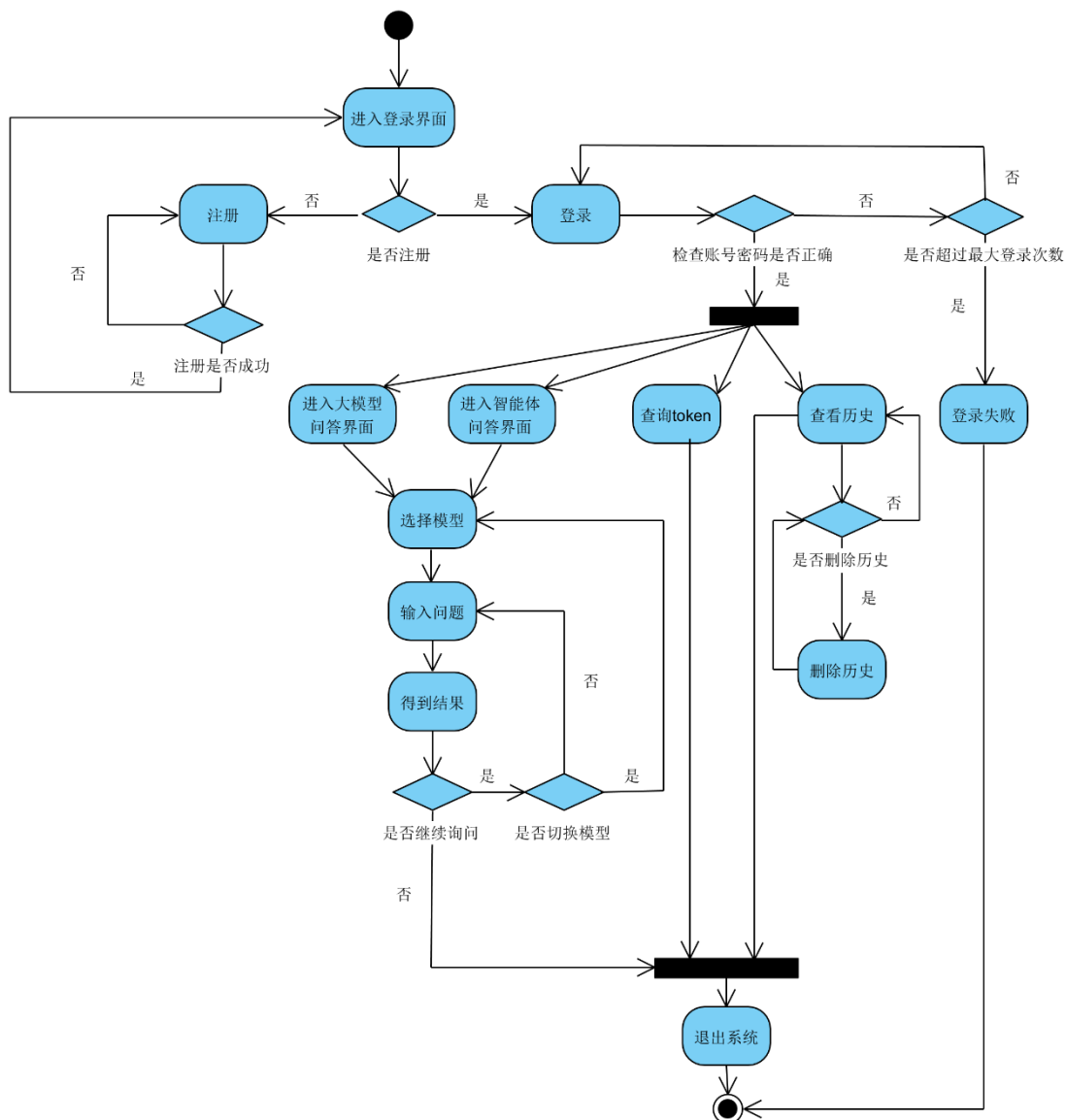


图 5-2 系统活动图

功能模块划分遵循高内聚低耦合原则。主要模块如图 5-4 所示，包括：用户认证与授权模块、智能体问答模块、大模型问答模块、历史记录管理模块、Token 管理模块、模型管理模块、用户管理模块、经验池管理模块、前端展示与路由模

块等。

其中智能体问答模块是本系统最主要的模块，其时序图如图 5-5 所示。该时序图详细展示了用户与智能体问答系统之间的完整交互流程，涉及五个主要参与对象：用户、智能体问答系统、数据库、LLM 系统和 LLM Server。

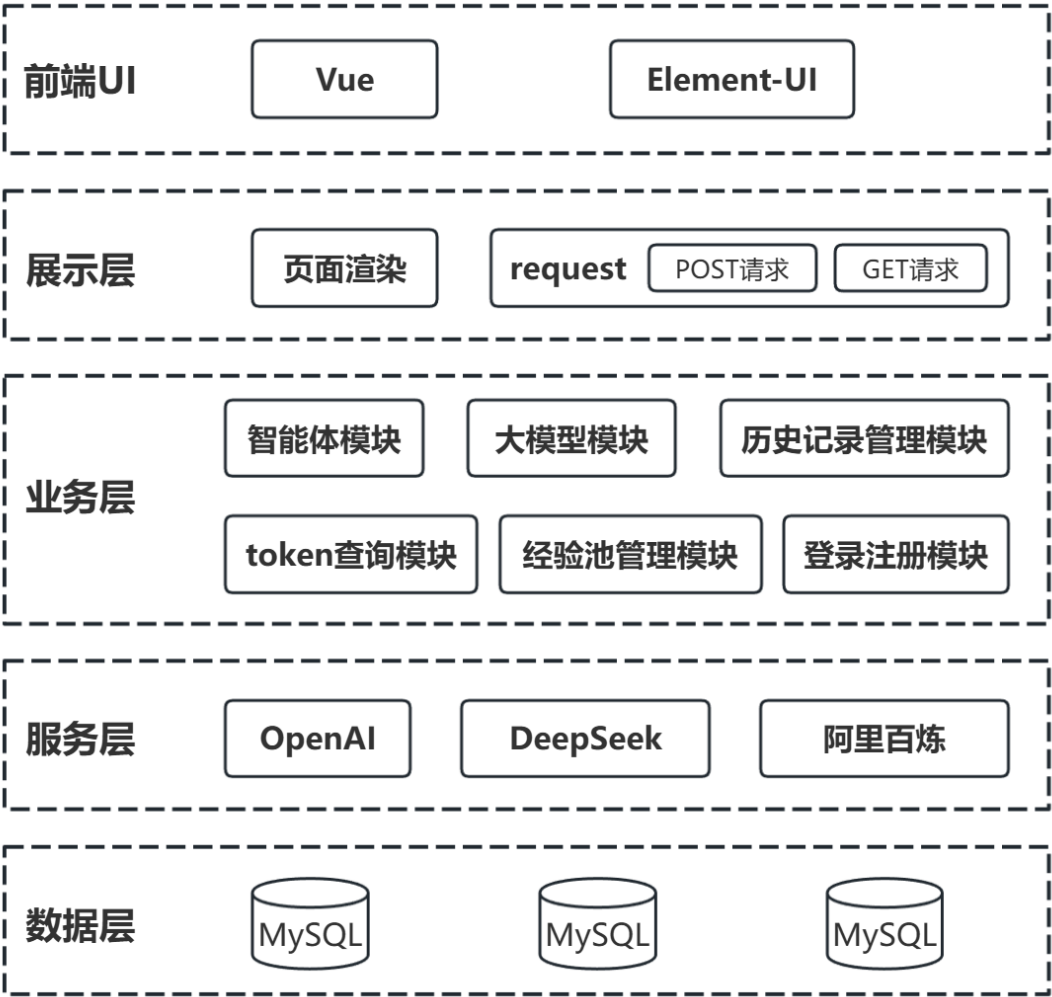


图 5-3 系统架构图

登录阶段展示了用户身份验证流程。用户发起登录请求后（消息 1），智能体问答系统首先向数据库查询用户信息（消息 1.1），数据库返回相应用户数据（消息 1.2）。验证通过后，系统向用户返回登录成功响应（消息 1.3），用户进入系统主界面。这个流程采用典型的请求-响应模式，体现了系统安全验证的基本机制。

模型更换阶段演示了系统配置变更过程。用户发起模型更换请求（消息 2），系统会查询数据库获取模型配置信息（消息 2.1），数据库返回模型详细信息（消息 2.2）。系统完成模型切换后向用户发送更换成功通知（消息 2.3）。该序列展示了系统如何动态加载不同大语言模型的配置参数，为后续问答服务提供基础支持。

问答处理阶段是整个时序图的核心部分，详细呈现了智能体问答的完整生命周期。用户输入问题（消息 3）后，系统触发一个复杂的处理链条：首先调用 API 准备工作（消息 3.1），这个步骤又包含三个子过程——获取模型信息、构建 prompt 和检索相关经验（消息 3.1.1），系统整合这些信息形成完整请求（消息 3.1.2）；接着向 LLM 服务商发起服务请求（消息 3.1.3），LLM Server 处理完成后返回响应（消息 3.1.4）；系统将处理结果返回给用户（消息 3.2，3.3）；最后异步存储问答记录（消息 3.4，3.5）。这个阶段展示了智能体系统的特色功能，包括经验检索与 prompt 优化，以及问题解答的完整数据流。

时序图通过纵向的时间维度和横向的对象协作维度，清晰地呈现了系统关键业务的执行顺序和组件交互。

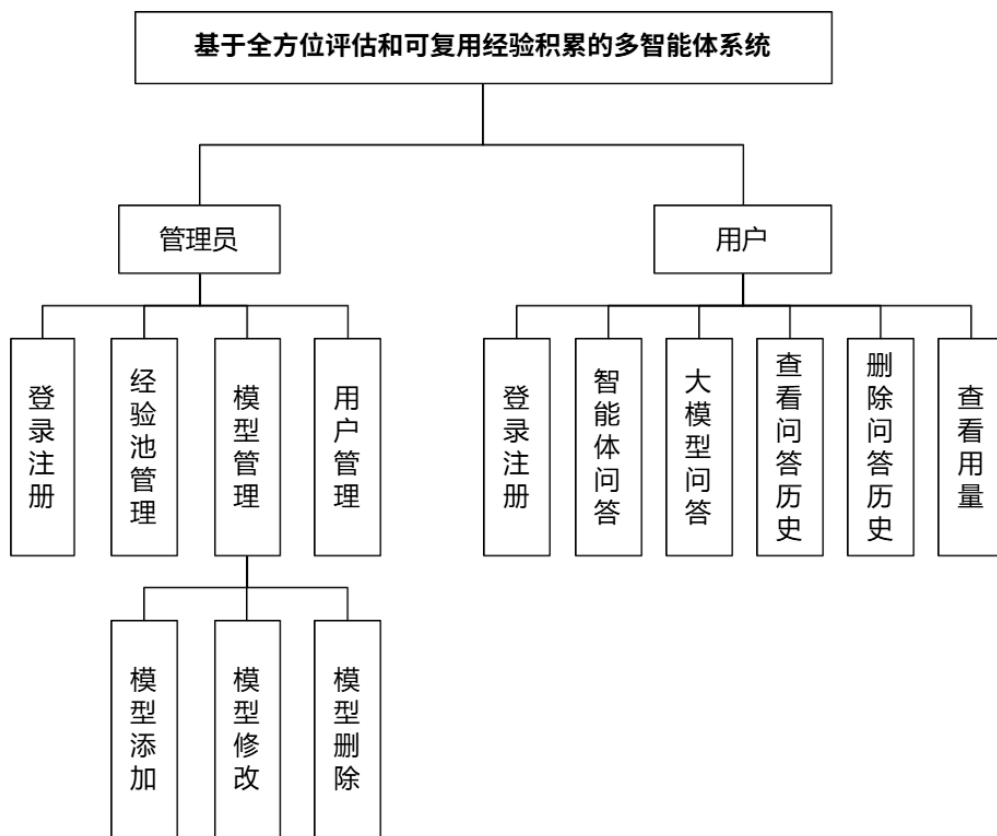


图 5-4 系统功能图

5.3 数据库设计

为实现多智能体系统的主要功能，本文采用 MySQL 数据库对数据进行存储。我们对数据库进行了设计，首先构建了系统的实体-关系图，如图 5-6 所示。

该 ER 图主要包含以下核心实体及其关系：

(1) 用户：包含用户 ID、名称和密码等属性，用于实现注册、登录、使用智能体和大模型。

(2) 智能体：由用户创建或使用，包含智能体 ID、名称、类型、描述等属性，可与工具、prompt、大模型、经验池关联。

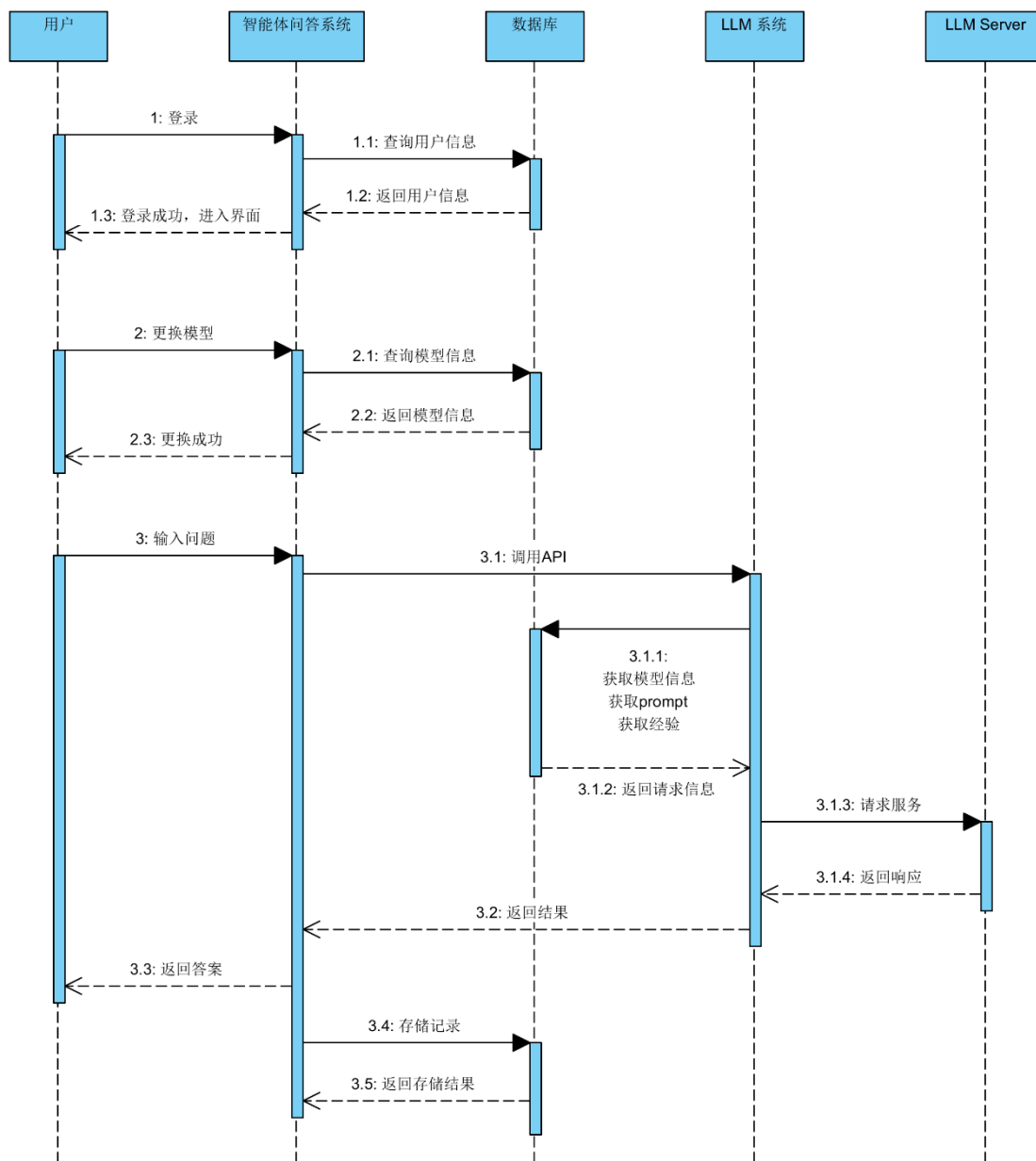


图 5-5 智能体问答模块时序图

(3) 大模型：系统支持多个大模型，包括大模型 ID、名称、API 接口、调用价格等属性，供智能体调用。

(4) 工具：智能体可结合外部工具调用，工具包含工具 ID、名称、API 与密钥信息等属性。

(5) **Prompt**: 智能体可绑定 prompt 进行任务提示, 系统支持 prompt 的版本控制。包含 ID、内容和类型等属性。

(6) **经验**: 智能体可以引用历史经验内容, 提高任务处理效果。包含经验 ID、内容和类型等属性。

(7) **历史记录**: 记录用户与智能体交互过程的内容和时间。包含历史 ID、内容和时间等属性。

(8) **管理员**: 用于后台管理系统资源, 如 prompt、模型、用户等。

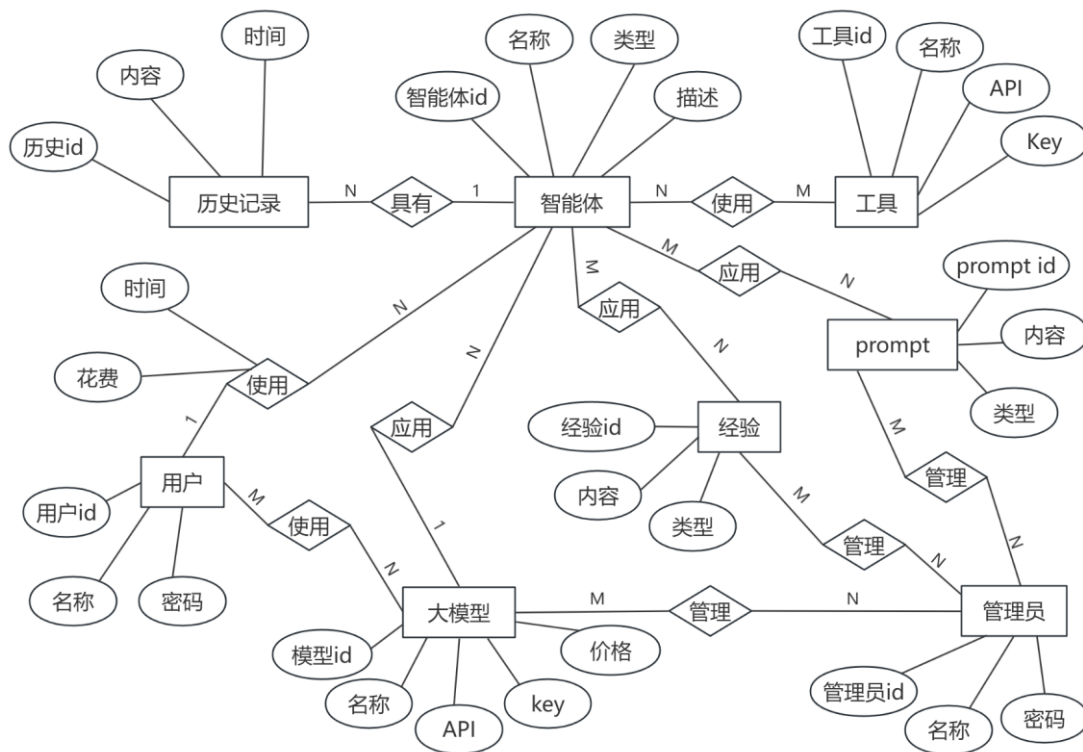


图 5-6 系统实体-关系图

在实体-关系图的基础上, 我们设计了数据库的概念数据模型、逻辑数据模型和物理数据模型, 如图 5-7, 图 5-8 和图 5-9 所示。

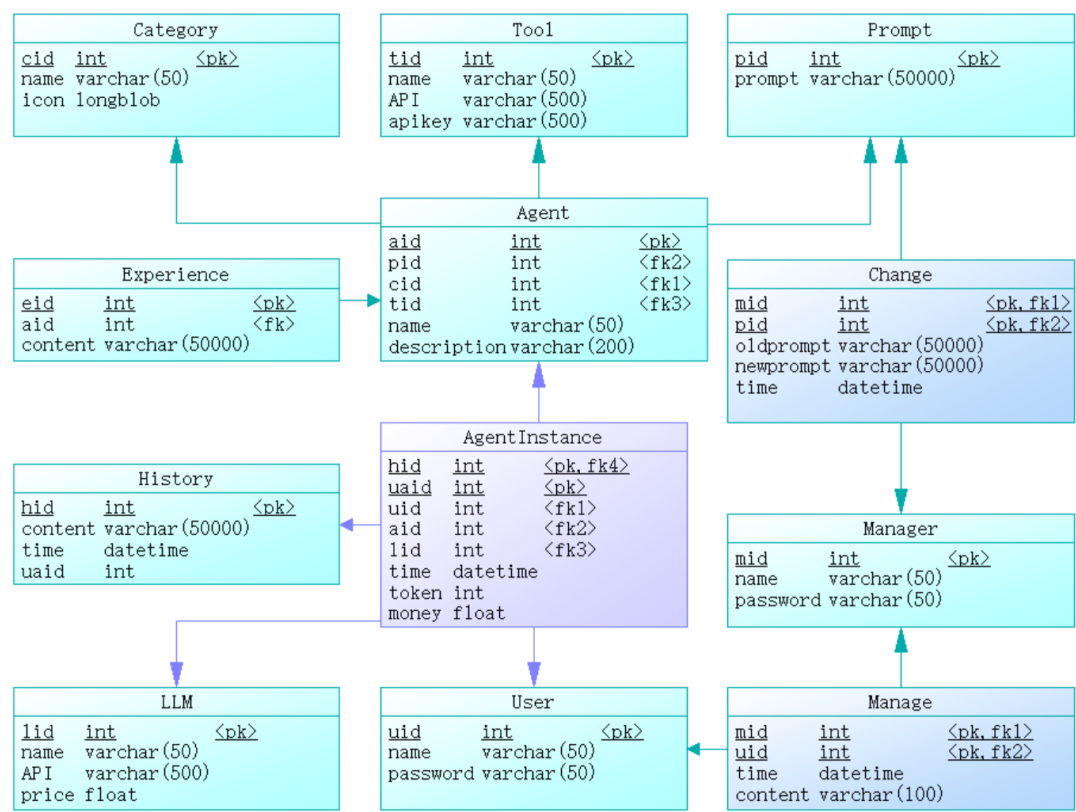


图 5-9 数据库物理模型

数据库物理模型图展示了多智能体应用系统的结构，设计上结构清晰、功能全面。**Agent** 表是整个系统的核心，定义了智能体的基本信息，包括其名称、描述以及关联的提示词、类别和工具。每个智能体通过外键与其他表建立联系。智能体的全局经验被记录在 **Experience** 表中，该表通过外键与 **Agent** 表连接，保存每个智能体在执行任务过程中的长文本经验内容，便于后续复用。

为了支持用户与智能体的实际交互，系统引入了 **AgentInstance** 表。该表记录了用户在某一时刻调用某个智能体的具体实例，并关联了所用的大语言模型和相关使用资源。每次用户与 **Agent** 的对话内容被保存到 **History** 表中，用于记录交互内容、时间以及关联的 **Agent** 实例编号。**Category** 表用于分类不同类型的智能体，字段包括分类名和图标，为系统提供便于用户浏览与查找的分类视图支持。

系统对大语言模型资源进行了抽象，使用 **LLM** 表记录不同语言模型的名称、调用 **API** 接口及价格信息。同时，通过 **Tool** 表维护 **Agent** 可使用的第三方工具资源，每个工具记录了其名称、接口地址和密钥信息。

为了提升提示词管理能力，系统设计了 **Prompt** 表来统一存储智能体初始提示词，并通过 **Change** 表记录管理员对提示词的历史修改。该表关联了 **Manager** 表中的管理员信息，追踪每次变更的管理员 **ID**、变更前的提示词和变更后的提

示词，确保提示词演化过程的可追溯性。

在权限管理方面，系统通过 **Manager** 表记录所有系统管理员的账户信息，并借助 **Manage** 表记录管理员对用户的操作日志，包括操作时间、具体内容和操作的用户 ID，增强系统的安全性与可监管性。

根据 ER 图和数据库模型对数据库进行开发，建立 **Mysql** 数据库和相应数据库表。其中智能体表和大模型表最为关键，其表结构如下所示：

表 5-7 智能体表结构

字段名	类型	描述
aid	int	主键，智能体 ID
name	varchar(50)	智能体名称
lid	int	外键，大模型 ID
category	varchar(50)	类别
description	varchar(200)	描述
uid	int	外键，用户 ID

表 5-8 大模型表结构

字段名	类型	描述
lid	int	主键，大模型 ID
name	varchar(50)	大模型名称
baseUrl	varchar(200)	请求地址
key	varchar(200)	密钥
inputPrice	float	输入价格
outputPrice	float	输出价格
uid	int	外键，用户 ID

5.4 系统实现

系统采用前后端分离模式开发。前端使用 **Vue 3** 编写组件逻辑并利用 **Element-Plus** 提供丰富的 UI 组件搭建页面。后端采用 **Spring Boot** 框架简化了企业级应用开发流程，通过 **Spring Security** 处理身份认证和授权。**MyBatis-Plus** 提供高效的数据库访问能力，系统使用 **MySQL 8.0** 作为存储数据库。开发环境具体版本表 5-9 所示。

表 5-9 开发环境

开发环境	名称
前端框架	Vue 3.5.13
后端框架	Spring Boot 3.4.4
数据持久化框架	MyBatis-plus 3.5.7
数据库	MySQL 8.0.31
开发语言	Java 17.0.2 Python 3.9.20
开发工具	IntelliJ IDEA 2022.2 Pycharm 2022.1

智能体应用系统的首页如图 5-10 所示，该页面显示了系统的标题和简要介绍。点击开始对话按钮，进入登录和注册界面，如图 5-11 所示。用户或管理员可以输入账号和密码进行登录。后端接收表单信息并进行验证，通过后则跳转至智能体空白界面，如图 5-12 所示。系统采用 Vue Router 来管理路由，通过 push 方法来实现路由跳转。

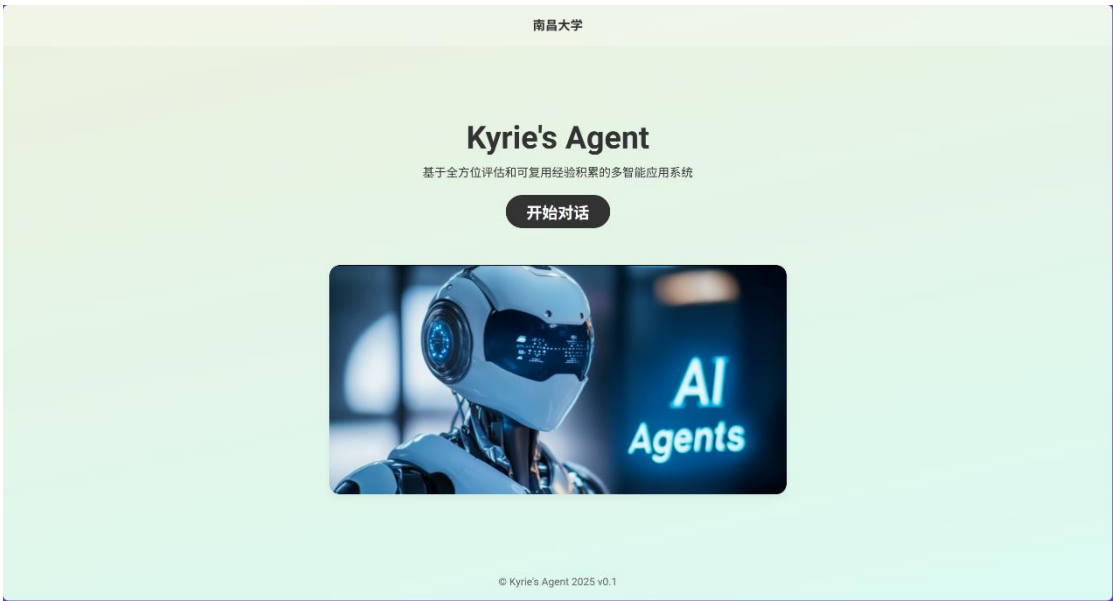


图 5-10 首页



图 5-11 登录界面

在智能体初始空白页，用户可以通过点击创意写作或旅行计划按钮来创建相应的智能体。创建智能体后，右侧子组件由空白界面跳转至问答界面。在问答界面用户可以根据智能体类型输入相应的问题。以旅行计划为例，用户可以输入目的地、时间、月份、季节、成员、兴趣爱好、预算等信息。如图 5-13 所示。

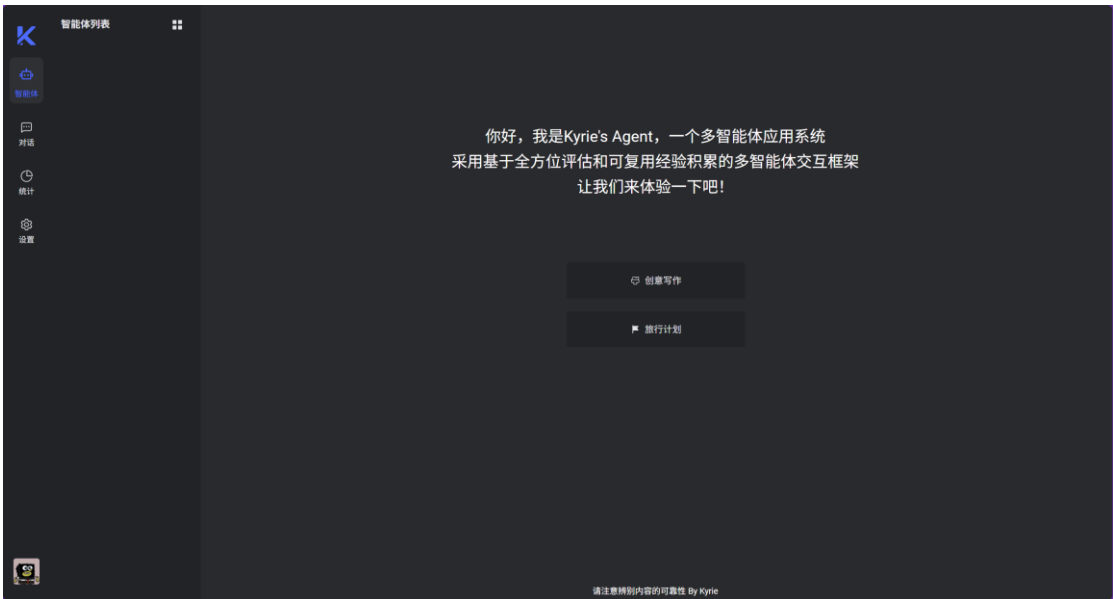


图 5-12 智能体空白界面



图 5-13 智能体问答界面

后端调用 LLM 服务，并运行基于多方位评估和可复用经验积累的多智能体算法框架。前端流式输出智能体内部讨论情况以及最终结果。智能体内部讨论情况如图 5-14 所示。最终结果如图 5-15 所示。使用智能体框架、调用 LLM、流式输出是本系统最核心且重点的工作，接下来将详细介绍该工作的核心代码。

当用户输入问题并点击发送按钮时，调用 `sendMessage` 方法，代码如下所示：

Code: sendMessage

```
const sendMessage = () => {
  const user_input = conv.value.inputText.trim();
  if (isStreaming.value) {
    ElMessage.error('请等待上一条消息处理完成');
    return
  }
  if (user_input) {
    isStreaming.value = true;
    appendUserMessage(user_input);
    forceScrollToBottom();
    const cur_res_id = conv.value.messages[conv.value.messages.length - 1].id;
    conv.value.inputText = '';
    fetchChatResponse(user_input, cur_res_id)
  } else {
    console.log('请输入消息');
```



```

}
}

```

该方法首先获取用户的输入，接着判断系统是否正在运行，如果运行则提醒用户等待上一条消息处理完成。若系统未运行，判断用户输入是否为空，若不为空，则系统开始处理信息。

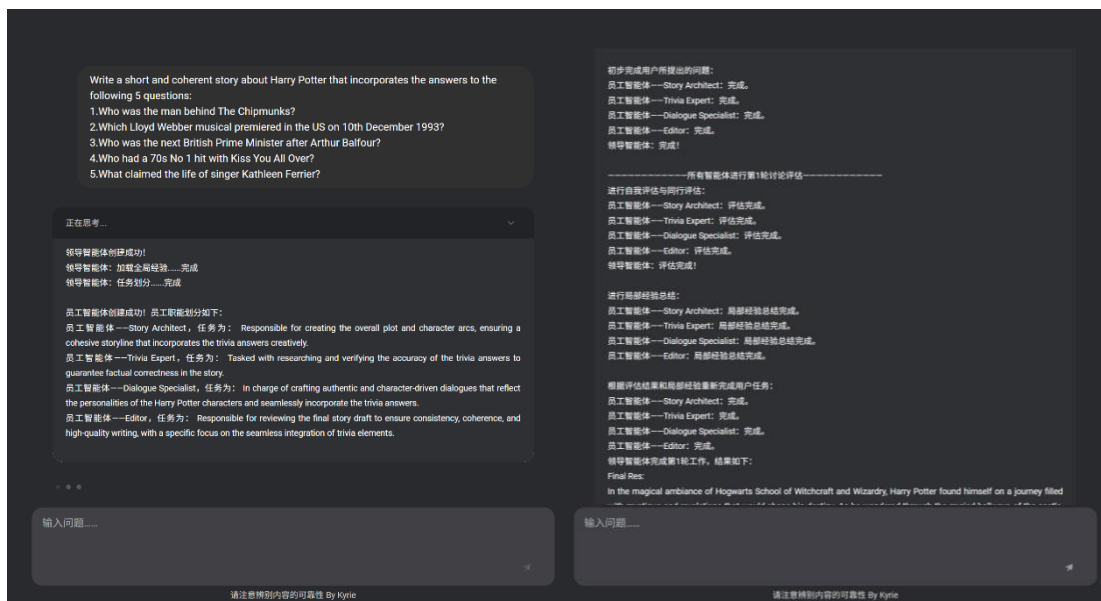


图 5-14 智能体内部讨论情况

系统首先将运行标志设为真，然后将用户输入添加到历史信息中，问答界面双向绑定，同步显示，并将页面滑至底端。获取当前消息的序号，并传值给 fetchChatResponse 方法。fetchChatResponse 方法如下所示：

Code: fetchChatResponse

```

const fetchChatResponse = (user_input, cur_res_id) => {
  .....
  .then((response) => {
    if (!response.body) throw new Error("ReadableStream not supported.");
    const reader = response.body.getReader();
    const decoder = new TextDecoder("utf-8");
    let buffer = "";
    const readChunk = () => {
      return reader.read().then(({ done, value }) => {
        if (done) {
          const msg = conv.value.messages.find((msg) => msg.id === cur_res_id)

```

```

        updateMessage({id: cur_res_id, status:"over"})
        conv.value.history.push({"role": "assistant", "content": msg.text})
        isStreaming.value = false;
        return;
    }
    buffer = decoder.decode(value, { stream: true });
    const lines = buffer.split('\n');
    for (let i = 0; i < lines.length ; i++) {
        const line = lines[i].trim();
        if (line) {
            try {
                const data = JSON.parse(line)
                if(data.choices.length == 0) {
                    continue
                }
                const reasoning_content = data.choices[0].delta.reasoning_co
ntent;

                const text = data.choices[0].delta.content;
                let status = "init";
                if(reasoning_content!=null && text==null){
                    status = "reasoning"
                }
                else if(reasoning_content==null && text!=null) {
                    status = "generating"
                }
            }
            .....
        }
    }

```

该方法通过 JSON 传值，获取到 response 响应后，判断响应体是否为空，为空则抛出异常。设置流读取器，创建 TextDecoder 用于解码 UTF-8 格式的流数据，初始化缓冲区 buffer 用于存储未处理完的数据。使用 reader.read() 异步读取数据块，然后对流数据进行处理。首先解码接收到的多行 JSON 数据为文本，然后按换行符分割，接着对每行数据进行 JSON 解析。当 reasoning_content 不为空时，系统为推理状态，智能体正在协商讨论；当 content 不为空时，系统为生成状态。通过以上代码实现了实时数据流处理，提高了用户体验。

fetchChatResponse 方法需要调用智能体运行 API，以获取 JSON 数据，具体代码如下：

Code: agentAPI

```

@app.post("/run_agent")
def agent(
    llm_name: str = Body(..., embed=True),
    task_class: str = Body(..., embed=True),
    question: str = Body(..., embed=True),
):
    headers = {
        "Content-Type": "text/event-stream; charset=utf-8",
        "Cache-Control": "no-cache",
        "Connection": "keep-alive",
    }
    exp, prompt, api, baseUrl = get_exp_prompt_llm(llm_name, task_class)
    llm = {
        "llm_name": llm_name,
        "base_url": baseUrl,
        "api": api
    }
    return StreamingResponse(run(question, task_class, exp, prompt, llm),
media_type='text/event-stream', headers=headers)

```

在上述代码中，该接口接收三个值，LLM 的名字、任务类别和任务。根据 LLM 名字和任务类别可以从数据库中获取智能体所需的经验、prompt、LLM 的请求地址和密钥等信息。通过 StreamingResponse，可以逐块发送数据给客户端，形成数据流。

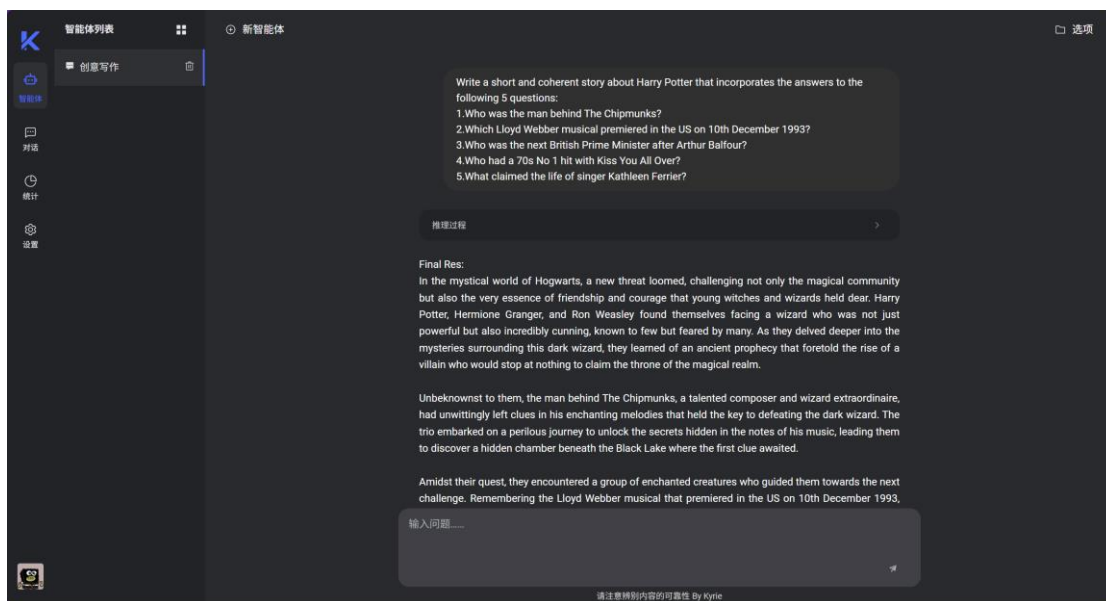


图 5-15 最终结果

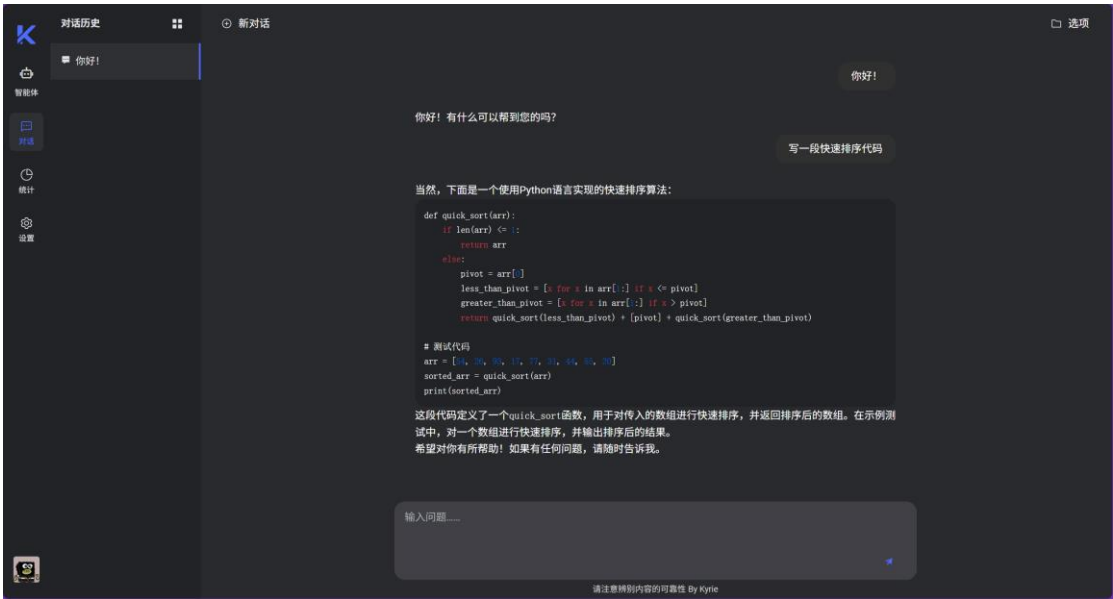
大模型对话界面如图 5-16 所示。系统构建了单一大模型对话功能，与 chatGPT 和 DeepSeek 网页版类似，可以读取用户的上下文，生成响应结果。不同的是，本系统支持随时更换大模型，可以对比不同大模型生成质量，也可以利用不同大模型的专长完成特殊任务。另外，本系统支持普通大模型和具有深思功能的大模型。

LLM 服务请求代码如下所示：

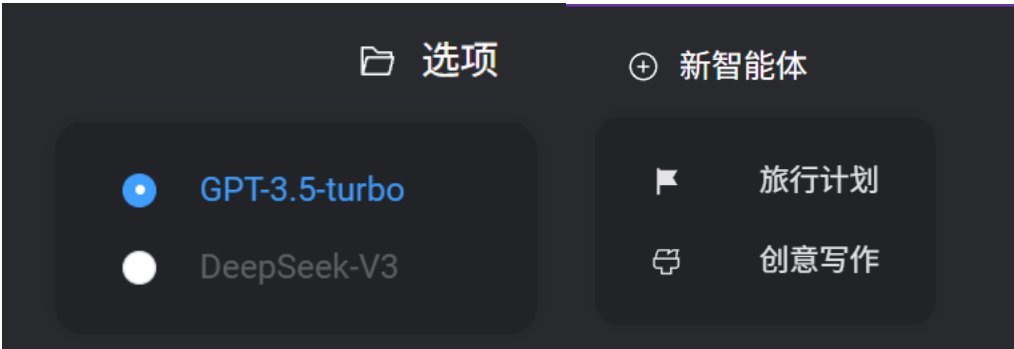
Code: requestLLM_stream

```
def requestLLM_stream(role_prompt, task, llm):
    api_key = llm['api']
    model = llm['llm_name']
    base_url = llm['base_url']
    while True:
        try:
            client = OpenAI(
                api_key=api_key,
                base_url=base_url,
            )
            completion = client.chat.completions.create(
                model=model,
                messages=[{'role': 'system', 'content': role_prompt},
                          {'role': 'user', 'content': task}],
                stream = True,
                stream_options={"include_usage": True}
            )
            return completion
        except Exception as e:
            print(f'An error occurred: {e}. Retrying...')
            time.sleep(1)
```

上述代码兼容 OpenAI 格式，只需传入 system 提示和 user 问题，以及 LLM 的名字、服务地址和密钥，即可调用 LLM，增强了方法的复用性。同时，可提供流式和非流式两种请求方式。



在智能体问答界面和大模型问答界面的右上角可以切换 LLM 服务，在智能体问答界面左上角可以创建新的智能体任务。效果如图 5-17 所示。



后端采用了 Spring Boot 的三层架构，本文以智能体经验和 prompt 获取为例进行介绍，代码如下：

```
Code: get_exp_prompt

def get_exp_prompt_llm(llm_name,task_class):
    data1 = {
        "llm_name": llm_name,
        "task_class": task_class
    }
    Response = requests.post('http://127.0.0.1:8081/agent/get_exp_prompt', json
=data1)
    response = json.loads(response.text).get("data")
```

```
exp = response.get("exp")
prompt = response.get("prompt")

return exp, prompt
```

上述代码通过发起后端 API 接口的 POST 请求，来获取经验和 prompt。在后端的三层架构中，Controller 层接收用户请求，对请求进行处理，并响应数据，代码如下所示：

Code: AgentController

```
@CrossOrigin
@RestController
@RequestMapping("/agent")
public class AgentController {
    @Autowired
    private ConfigureService configureService;
    @PostMapping("/get_exp_prompt")
    public Result get_exp_and_promt(@RequestBody Agent agent){
        String task_class = agent.getTask_class();
        Map<String,String> prompts = configureService.get_prompt(task_class);
        List<String> exps = configureService.get_exp(task_class);
        agent.setExp(exps);
        agent.setPrompt(prompts);
        return Result.success(agent);
    }
}
```

Controller 层调用 Service 层进行逻辑处理，Service 层用于处理具体的业务逻辑，其接口和实现方法如下所示：

Code: ConfigureService

```
public interface ConfigureService {
    public Map<String, String> get_prompt(String type);
    public List<String> get_exp(String type);
}
```

Code: ConfigureServiceImpl

```
@Service
```

```

public class ConfigureServiceImpl implements ConfigureService {
    @Autowired
    PromptMapper promptMapper;
    @Autowired
    ExperienceMapper experienceMapper;
    @Autowired
    LLMMapper llmMapper;
    public Map<String, String> get_prompt(String type){
        List<Prompt> prompts = promptMapper.get_prompt(type);
        Map<String, String> map = new HashMap<String, String>();
        for(int i=0; i<prompts.size();i++){
            map.put(prompts.get(i).getPname(), prompts.get(i).getPrompt());
        }
        return map;
    }
    public List<String> get_exp(String type){
        List<String> exp = experienceMapper.get_exp(type);
        return exp;
    }
}

```

Service 层调用 Mapper 层，从 Mapper 层获取逻辑处理过程中需要的数据。Mapper 层负责定义接口和数据操作的类型，mapper.xml 负责具体的 SQL 实现，相关代码如下：

Code: PromptMapper 和 ExperienceMapper

```

@Mapper
public interface PromptMapper {
    public List<Prompt> get_prompt(String type);
}

@Mapper
public interface ExperienceMapper {
    public List<String> get_exp(String type);
}

```

Code: PromptMapper.xml 和 ExperienceMapper.xml

```

<?xml version="1.0" encoding="utf-8" ?>

```

```
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.example.agent_spring.mapper.ExperienceMapper">
    <select id="get_exp" parameterType="String" resultType="String">
        select content from experience where type = #{type}
    </select>
</mapper>

<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.example.agent_spring.mapper.PromptMapper">
    <select id="get_prompt" parameterType="String" resultType="com.example.agent_spring.entity.Prompt">
        select Pname,prompt from prompt where Ptype = #{type}
    </select>
</mapper>
```

Mapper 层操作数据库，拿到的数据后返回给 Service 层。Service 层处理完后，把处理结果返回给 Controller 层，Controller 层将结果响应给用户。至此，后端完成了三层架构的数据传送与处理。通过这种架构，可以将表现层、业务层和数据层分离，提高代码的可维护性和可扩展性。

第六章 总结与展望

6.1 工作总结

随着人工智能技术的不断发展,单一大模型或智能体在面对复杂多变的任务时表现出局限性,难以满足多角色协作、高效率处理等实际需求。多智能体系统作为一种新型的智能协作范式,逐渐展现出强大的任务适应能力和广泛的应用前景。尤其是在大语言模型快速发展的推动下,多智能体系统具备了更强的自然语言理解与交互能力,在创意写作、旅行规划等复杂任务中展现出良好的性能。针对现有多智能体系统中经验复用能力弱、评估机制单一等问题,本文所做工作如下:

(1) 提出了一种基于多方位评估和可复用经验积累的多智能体协作框架。在多智能体系统中引入类似公司绩效考核的方法,将评价分为多层次互动环节。每个智能体首先进行自评,审视自身输出质量。然后接受同级其他智能体的互评,从多角度给出反馈。最后由上级智能体进行综合评估。这一机制将多种评估视角结合,可多角度衡量结果的质量和团队协作效果。同时,提出了一种双层经验池机制,帮助智能体在处理复杂任务时收集和积累有价值的经验。具体而言,一层为全局经验池,用于存储所有任务的成功经验。当新任务到来时,智能体可从全局经验池中总结先前相似任务的经验;另一层为局部经验池,为每个智能体存放本次任务历史轮次中的经验。双层经验池可兼顾集体智慧与个体专长,提高经验的复用能力和任务的泛化能力。通过实验验证,该框架在两个典型任务上均表现出优异性能,超越了多种先进大模型及多智能体基线方法,具备良好的实用性与拓展性。

(2) 基于本文提出的多智能体协作框架,本文采用 Vue、SpringBoot 和 MySQL,设计并开发了一个稳定、高效且易于使用的多智能体应用系统。系统采用前后端分离开发,实现多智能体应用、大模型问答、智能体管理、经验池管理、修改 Prompt 提示词等功能。多智能体应用部分设计了旅行计划、创意写作等模块。对于旅行计划,我们利用智能体分析用户偏好,提供个性化的景点、住宿和活动建议,同时考虑交通状况、天气变化等因素为用户提供最优路径。针对创意写作,智能体将帮助用户打破创作瓶颈,通过提供故事线、角色设定或情节发展来激发创造力。

6.2 未来展望

本文提出的基于多方位评估与可复用经验积累的多智能体协作框架在多个数据集上达到了理想的效果，多智能体应用系统也展现出良好的实用性，但仍存在一些局限和可优化空间，具体如下：

（1）目前可用于复杂任务求解的公开数据集较为有限，难以覆盖多种真实应用场景，限制了本文所提出的框架在更大范围的任务中的泛化能力验证。未来可以考虑构建或引入更具挑战性和多样性的复杂任务数据集，更全面地评估系统在实际复杂环境下的表现与适应性。

（2）本文目前聚焦于文本输入与输出的任务，尚未涉及多模态数据的处理。随着多模态大模型的发展，未来可引入图像、语音、视频等多模态数据作为输入信息，赋予多智能体更强的感知能力与决策能力，从而拓展系统在实际应用中的适用场景和任务边界。

（3）在系统功能方面，未来可以通过图形界面形式呈现各个智能体的任务分工、交互路径与评估反馈，使用户能够更直观地理解协作机制与决策逻辑。同时，可扩展更复杂的任务流程与工具调用能力，进一步增强系统的通用性与灵活性，为实际部署和应用提供更强支撑。

参考文献

- [1] Minsky M. Society of mind[M]. Simon and Schuster, 1988.
- [2] Achiam J, Adler S, Agarwal S, et al. Gpt-4 technical report[J]. arXiv preprint arXiv:2303.08774, 2023.
- [3] Guo D, Yang D, Zhang H, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning[J]. arXiv preprint arXiv:2501.12948, 2025.
- [4] Hong S, Zhuge M, Chen J, et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework[C] The Twelfth International Conference on Learning Representations, 2023.
- [5] Qian C, Cong X, Yang C, et al. Communicative Agents for Software Development[J]. CoRR, 2023.
- [6] Zheng Y, Ma C, Shi K, et al. Agents meet okr: An object and key results driven agent system with hierarchical self-collaboration and self-evaluation[J]. arXiv preprint arXiv:2311.16542, 2023.
- [7] Liu Z, Zhang Y, Li P, et al. Dynamic LLM-Agent Network: An LLM-agent Collaboration Framework with Agent Team Optimization[J]. CoRR, 2023.
- [8] Wang Y, Kordi Y, Mishra S, et al. Self-Instruct: Aligning Language Models with Self-Generated Instructions[C] Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2023: 13484-13508.
- [9] Shinn N, Cassano F, Gopinath A, et al. Reflexion: Language agents with verbal reinforcement learning[J]. Advances in Neural Information Processing Systems, 2023, 36: 8634-8652.
- [10] Xi Z, Jin S, Zhou Y, et al. Self-Polish: Enhance Reasoning in Large Language Models via Problem Refinement[C] The 2023 Conference on Empirical Methods in Natural Language Processing, 2023.
- [11] Nilsson N J. Logic and artificial intelligence[J]. Artificial intelligence, 1991, 47(1-3): 31-56.
- [12] 于江涛. 多智能体模型, 学习和协作研究与应用[D]. 杭州: 浙江大学, 2003.
- [13] Leng Y, Yuan Y. Do LLM Agents Exhibit Social Behavior?[J]. arXiv preprint arXiv:2312.15198, 2023.
- [14] Team G, Anil R, Borgeaud S, et al. Gemini: a family of highly capable multimodal models[J]. arXiv preprint arXiv:2312.11805, 2023.
- [15] Roulmeliotis K I, Tselikas N D. Chatgpt and open-ai models: A preliminary review[J]. Future Internet, 2023, 15(6): 192.
- [16] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[J]. Advances in neural information processing systems,

- 2022, 35: 24824-24837.
- [17] Yao S, Yu D, Zhao J, et al. Tree of thoughts: Deliberate problem solving with large language models[J]. Advances in neural information processing systems, 2023, 36: 11809-11822.
- [18] Besta M, Blach N, Kubicek A, et al. Graph of thoughts: Solving elaborate problems with large language models[C]. Proceedings of the AAAI Conference on Artificial Intelligence. 2024, 38(16): 17682-17690.
- [19] Yao S, Zhao J, Yu D, et al. React: Synergizing reasoning and acting in language models[C]. International Conference on Learning Representations (ICLR). 2023.
- [20] Yin Z, Sun Q, Chang C, et al. Exchange-of-Thought: Enhancing Large Language Model Capabilities through Cross-Model Communication[C] Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics (ACL), 2023.
- [21] Tran K T, Dao D, Nguyen M D, et al. Multi-Agent Collaboration Mechanisms: A Survey of LLMs[J]. CoRR, 2025.
- [22] Venkatraman S, Tripto N I, Lee D. Collabstory: Multi-llm collaborative story generation and authorship analysis[J]. In Findings of the Association for Computational Linguistics: NAACL 2025, 2025.
- [23] Yu X, Fu J, Deng R, et al. MineLand: Simulating Large-Scale Multi-Agent Interactions with Limited Multimodal Senses and Physical Needs[J]. CoRR, 2024.
- [24] Xu C. 基于大语言模型的自主智能体概述 (A Survey on Large Language Model based Autonomous Agents Xu Chen)[C]. Proceedings of the 23rd Chinese National Conference on Computational Linguistics (Volume 2: Frontier Forum). 2024: 141-150.
- [25] Zhang Y, Sun R, Chen Y, et al. Chain of agents: Large language models collaborating on long-context tasks[J]. Advances in Neural Information Processing Systems, 2024, 37: 132208-132237.
- [26] Chen W, Su Y, Zuo J, et al. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors[C] The Twelfth International Conference on Learning Representations, 2023.
- [27] Wang Z, Mao S, Wu W, et al. Unleashing cognitive synergy in large language models: A task-solving agent through multi-persona selfcollaboration[J]. In Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2024.
- [28] Chen G, Dong S, Shu Y, et al. AutoAgents: a framework for automatic agent generation[C] Proceedings of the Thirty-Third International Joint Conference on

- Artificial Intelligence. 2024: 22-30.
- [29] Gandhi S, Shah D, Patwardhan M, et al. ResearchCodeAgent: An LLM Multi-Agent System for Automated Codification of Research Methodologies[J]. arXiv preprint arXiv:2504.20117, 2025.
- [30] Chan C M, Chen W, Su Y, et al. ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate[C] The Twelfth International Conference on Learning Representations, 2023.
- [31] Liang T, He Z, Jiao W, et al. Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate[C] Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024: 17889-17904.
- [32] Bo X, Zhang Z, Dai Q, et al. Reflective multi-agent collaboration based on large language models[J]. Advances in Neural Information Processing Systems, 2024, 37: 138595-138631.
- [33] Cemri M, Pan M Z, Yang S, et al. Why Do Multi-Agent LLM Systems Fail?[J]. ICLR 2025 Workshop on Building Trust in Language Models and Applications, 2025.
- [34] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.
- [35] Shao M, Basit A, Karri R, et al. Survey of different large language model architectures: Trends, benchmarks, and challenges[J]. IEEE Access, 2024.
- [36] Kenton J D M W C, Toutanova L K. Bert: Pre-training of deep bidirectional transformers for language understanding[C]. Proceedings of naacL-HLT. 2019, 1(2).
- [37] Sun Y, Wang S, Li Y, et al. Ernie: Enhanced representation through knowledge integration[J]. arXiv preprint arXiv:1904.09223, 2019.
- [38] Lan Z, Chen M, Goodman S, et al. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations[C] International Conference on Learning Representations, 2019.
- [39] Grattafiori A, Dubey A, Jauhri A, et al. The llama 3 herd of models[J]. arXiv preprint arXiv:2407.21783, 2024.
- [40] Ren X, Zhou P, Meng X, et al. Pangu- Σ : Towards trillion parameter language model with sparse heterogeneous computing[J]. arXiv preprint arXiv:2303.10845, 2023.
- [41] Brown T, Mann B, Ryder N, et al. Language models are few-shot learners[J]. Advances in neural information processing systems, 2020, 33: 1877-1901.
- [42] Team G, Anil R, Borgeaud S, et al. Gemini: a family of highly capable multimodal models[J]. arXiv preprint arXiv:2312.11805, 2023.

- [43] Liu A, Feng B, Xue B, et al. Deepseek-v3 technical report[J]. arXiv preprint arXiv:2412.19437, 2024.
- [44] Yang A, Zhang B, Hui B, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement[J]. arXiv preprint arXiv:2409.12122, 2024.
- [45] Sun Y, Wang S, Feng S, et al. Ernie 3.0: Large-scale knowledge enhanced pre-training for language understanding and generation[J]. arXiv preprint arXiv:2107.02137, 2021.
- [46] Xi Z, Chen W, Guo X, et al. The rise and potential of large language model based agents: A survey[J]. Science China Information Sciences, 2025, 68(2): 121101.
- [47] Xie J, Zhang K, Chen J, et al. TravelPlanner: A Benchmark for Real-World Planning with Language Agents[C] International Conference on Machine Learning. PMLR, 2024: 54590-54613.

致谢

行文至此，好像为过去的四年画上了句号，是结束，亦是开始。

感谢我的指导老师黄旭慧教授和田植良教授。黄老师曾多次指导我参加学科竞赛，带领我获得了多次国家级和省级奖项。在本次毕业设计的完成过程中，黄老师为我提出了许多建设性意见和指导。田老师从选题立意到写作技巧，都悉心教诲我，耐心为我答疑解惑。感谢两位导师对我的指导和帮助，顺利完成了本毕业设计。

感谢我的父母和家人，一直对我无私的付出和关爱。在我的求学之路上，给予我最坚实的后盾和勇敢面对挑战的底气。

感谢我的辅导员江宁老师和熊伟老师，在学习和生活上给予我的关心和帮助；感谢李向军老师和刘伯成老师，在科研和竞赛方面对我的鼓励和指导；感谢黄安老师和江婕书记，对我的教诲和认可。

感谢我的同学和朋友们，在我失意时给予我安慰与陪伴，在我迷茫时给予我温暖与鼓励。山水一程，三生有幸，让我在求学路上不再孤单。

感谢所有我爱的和爱我的人。

人生漫漫，山高路远，来日方长，后会有期！